



Applied Machine Learning

In-Depth Companion Guide

A visual, mathematical and intuition-first journey

Course Title	Applied Machine Learning
Course Code	SE ZG568 / SS ZG568
Credit Model	3 – 1 – 0 (4 credits)
Programme	BITS Pilani — WILP
Lead Instructor	Saurabh
Author	Saurabh (companion document for students)
Coverage	Sessions 9 – 15

“ The aim is not to memorise algorithms, but to see them — to feel what each one is doing on real data. ”

Topics covered: Decision Trees · Entropy & Information Gain · Bias-Variance · Bagging · Boosting · Random Forest · PCA · K-Means · DBSCAN · Artificial Neural Networks · Forward & Back Propagation · Activation Functions · Vanishing Gradient · Regularisation · CNNs (with stride / padding examples) · RNNs · Encoder-Decoder · Attention · Bias & Fairness · Interpretable ML.



How to Use This Guide

This document is the companion to the second half of **SE ZG568 — Applied Machine Learning**. It maps each contact session to a chapter, and each chapter follows the same recipe. The goal is depth without dryness: every algorithm is introduced with an intuition, illustrated with a picture, then formalised with math, and finally walked through with a worked numerical example.

Read it cover-to-cover the first time. After that, treat it as a reference — every section is self-contained.

The recipe

1. Core Idea	A one-sentence summary that gives you the big picture.
2. Visual Intuition	A diagram that says everything the equation will say later — but in pictures.
3. The Math	Formal equations with each symbol translated into plain English.
4. Worked Example	Numbers worked out by hand, step by step, so you can follow without doubt.
5. Analogy	A real-world story that makes the idea stick when you forget the math.
6. When to Use	Practical advice on when each technique shines and when it fails.
7. Common Pitfalls	Mistakes students (and engineers) routinely make — labelled before you make them.

Notation we will use throughout

$\mathbf{x}$	an input feature or feature vector — the raw measurement (a pixel, a word, a sensor reading)
y	the true label or target value — what we are trying to predict
$\hat{y}$	the model's prediction — what the model guessed
n	the number of training samples
$\mathbf{d}$ or $\mathbf{p}$	the number of features (the input dimension)
$\mathbf{W}, \mathbf{b}$	weight matrix and bias vector of a layer
$\sigma(\mathbf{z})$	a non-linear activation function applied element-wise
L	the loss function — what we are trying to minimise
η	the learning rate — how big each gradient-descent step is
$\partial L / \partial \mathbf{w}$	partial derivative of the loss with respect to weight $\mathbf{w}$
$E[\cdot]$	expected value (long-run average over the data distribution)
$\mathbf{1}[\cdot]$	the indicator function — 1 if the condition inside is true, 0 otherwise

How to read math without fear. Every formula in this guide is followed by a plain-English translation. If a formula intimidates you, do not skip it — read the line right after it; you will discover the formula was just a compact summary of the picture you already saw.



Contents

This guide follows the official Applied Machine Learning syllabus (SE ZG568, Sessions 9 – 15). Each session is one chapter; each chapter stands on its own.

How to Use This Guide	2
The recipe	2
Notation we will use throughout	3
Contents	4
Decision Trees	8
9.1 Why study decision trees first	8
9.2 Core idea	8
9.3 Measuring uncertainty: entropy and Gini	9
Entropy — definition and intuition	9
Gini impurity	10
9.4 Information gain — a 'good' split, defined	10
9.5 Worked example — information gain by hand	11
9.6 Construction algorithm (ID3 / CART)	12
9.7 Bias-Variance Tradeoff in Decision Trees	12
The decomposition formally	13
9.8 When to use, and when not to	14
9.9 Common pitfalls	14
Ensembles — When Many Average Models Beat One Genius	17
10.1 The motivation, in plain English	17
10.2 Why averaging actually reduces variance	17
10.3 Bagging — Bootstrap Aggregating	19



Why bootstrap sampling? The 63% rule	19
Worked numbers — variance reduction	19
10.4 Boosting — fix the previous model's mistakes	21
The tutor analogy	21
AdaBoost — the original algorithm in detail	21
Modern boosting — gradient boosting and XGBoost	22
10.5 Bagging vs Boosting at a glance	23
10.6 Random Forest — bagging on steroids	24
Why this works — the doctor analogy	24
Hyperparameters that actually matter	24
Feature importance — a free bonus	25
10.7 When to use which ensemble	25
10.8 Choosing between forests and neural networks	25
Unsupervised Learning — Finding Structure Without Labels	27
11.1 Why unsupervised learning matters	27
11.2 Principal Component Analysis (PCA)	28
Geometric intuition — three pictures in one	28
The math, in four short steps	28
How many components to keep?	29
Worked example — a tiny 2-D dataset	29
Practical tips	30
11.3 K-Means Clustering	31
The objective and algorithm	31
Choosing K — elbow and silhouette	31
K-Means++ initialisation	32
Limitations of K-Means	32
11.4 DBSCAN — Density-Based Clustering	33
Two hyperparameters — and that is it	33



The algorithm in pseudocode	33
Choosing ϵ — the k-distance plot	34
11.5 K-Means vs DBSCAN — when to choose which	34
Artificial Neural Networks — From One Neuron to a Network	36
12.1 Why neural networks	36
12.2 The Single Neuron — Perceptron	36
Boolean logic with one neuron	37
12.3 The Multi-Layer Perceptron (MLP)	38
12.4 Forward Pass — worked by hand on Figure 12.2	38
12.5 Loss functions and gradient descent	40
12.6 Backpropagation in plain English	40
12.7 Why Networks Can Learn (Almost) Anything	41
12.8 Why depth, specifically — the hierarchical-features story	41
Deep Learning — Networks With Many Hidden Layers	42
13.1 What 'deep' actually means	42
13.2 Activation functions — the intuition	43
The five activations to know	43
13.3 The Vanishing Gradient Problem	45
Other defences against vanishing gradient	45
13.4 Regularisation — fighting overfitting	46
L2 — Ridge / weight decay	46
L1 — Lasso	46
Dropout — the most elegant regulariser ever invented	46
Early stopping & data augmentation	47
Convolutional, Recurrent and Attention-Based Networks	48



14.1 The motivation	48
14.2 Convolutional Neural Networks (CNNs)	49
Why convolutions — two big intuitions	49
The math of one convolution cell	49
Output-size formula — the most important formula in CNNs	50
Six solved examples — get fluent with the formula	50
Pooling — the friendly down-sampler	51
Putting it all together — a tiny CNN classifier	52
14.3 Recurrent Neural Networks (RNNs)	53
The recurrence equations	53
LSTM and GRU — solving long-term dependency	53
14.4 Encoder - Decoder Architectures (seq2seq)	55
The bottleneck problem	55
14.5 The Attention Mechanism	55
The math, in three lines	56
FAccT — Bias, Fairness and Interpretability	57
15.1 Why this matters now	57
15.2 Where bias comes from	58
15.3 Fairness — there is no single definition	58
15.4 Mitigating bias	59
15.5 Interpretable Machine Learning	60
Two families of methods	60
LIME — local linear approximation	60
SHAP — fair credit via game theory	60
Other useful tools you should know	61
15.6 Interpretability in high-stakes domains	61
Closing Notes for Students	63

CONTACT SESSION 9

Decision Trees, Entropy & Information Gain

Decision Trees

9.1 Why study decision trees first

Out of every machine-learning model ever invented, the decision tree is the closest to how humans naturally think. We don't internally compute weighted averages of inputs the way a linear model does — we ask questions. “*Is it raining?*”, “*Did I have breakfast?*”, “*Did the customer pay last time?*” — and we follow the answers down a flowchart of further questions until we reach a decision. Decision trees formalise that.

Three reasons every ML practitioner needs to know trees deeply:

- **They are interpretable.** You can literally *print* a decision tree and a non-technical stakeholder will understand it. In regulated domains (medicine, credit, hiring) this is non-negotiable.
- **They are the building block of the strongest classical models.** Random Forest, XGBoost, LightGBM and CatBoost — all dominate Kaggle and industry — are ensembles of trees.
- **They reveal the bias-variance tradeoff in its purest form.** Adjust one knob (depth) and you can dial smoothly from extreme bias to extreme variance.

“If outlook is sunny — and humidity is high — then probably no tennis today.” That is a decision tree. The trick is figuring out, automatically, which questions to ask first.

9.2 Core idea

In one sentence. A decision tree learns by repeatedly splitting the data with the question that *most reduces uncertainty* about the label, and stopping when each leaf is pure (or close to pure).

A decision tree learned for the 'play tennis' dataset

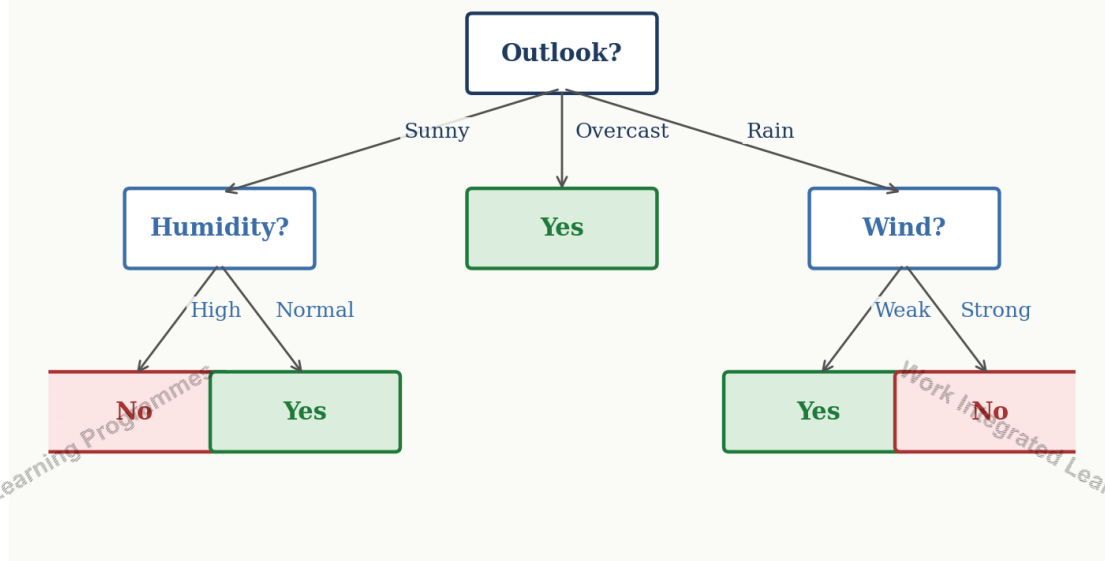


Figure 9.1 — A decision tree learned on the classic 'play tennis' dataset. Internal nodes ask questions; leaves produce predictions.

Walk Figure 9.1 yourself. Start at the root: the algorithm has decided the most informative first question is 'What is the outlook?'. If the answer is *Overcast*, we are done — we predict **Yes**. If the answer is *Sunny* we ask a follow-up about humidity; if *Rain* we ask about wind. Each path from root to leaf is a human-readable rule. That readability is the magic of decision trees.

9.3 Measuring uncertainty: entropy and Gini

Suppose a node holds 9 spam emails and 5 ham emails. We want a single number that says "how mixed up is this node?". A pure node (all spam, or all ham) should score 0; a perfectly mixed node (50/50) should score the maximum. Two classical measures fit this bill: **entropy** (from information theory, in bits) and **Gini impurity** (from economics, originally a measure of inequality).

Entropy — definition and intuition

For a node containing K classes with proportions $p_1, \dots, p_K$, entropy is:

$$H(\text{node}) = - \sum_{k=1..K} p_k \cdot \log_2(p_k)$$

This formula has a beautiful coding-theory interpretation. Suppose I want to send you the class of a random point from this node, and we agree on an optimal binary code in advance. Entropy is the *average number of bits* I would need to send. If the node is pure (one class), I need zero bits (you already know the answer). If it is 50/50, I need exactly one bit. If it is 75/25 — somewhere in between (about 0.81 bits): the more skewed, the less info I need.

Gini impurity

Gini measures the probability that two random draws from the node would land in different classes:

$$\text{Gini}(\text{node}) = 1 - \sum_k p_k^2$$

Same shape as entropy: 0 when pure, maximum (= 0.5 for binary) at the most-mixed point. In practice, the choice between Gini and entropy almost never affects the final tree — Gini is a touch cheaper to compute (no logarithms), so libraries default to it.

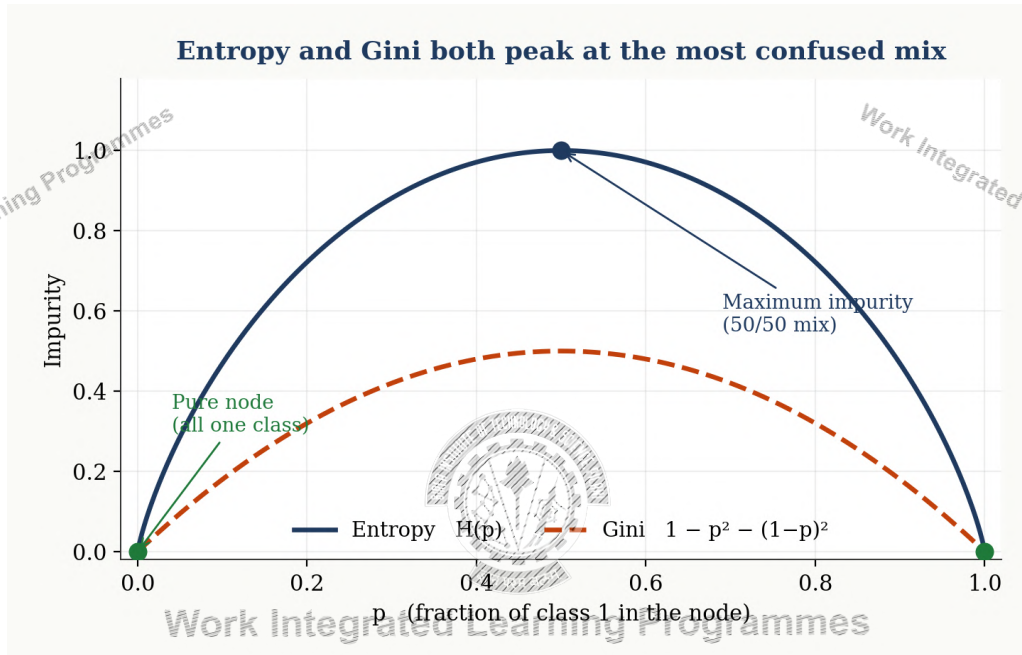


Figure 9.2 — Entropy and Gini have the same hump shape: both are zero at $p=0$ or $p=1$ (pure), maximal at $p=0.5$ (the most confused mix).

9.4 Information gain — a 'good' split, defined

When we split a node into children using some feature, we hope the children are *purer* than the parent. The drop in (weighted) impurity is called **information gain**:

$$\text{IG}(\text{split}) = H(\text{parent}) - \sum_c (n_c / n) \cdot H(\text{child}_c)$$

In English: take the parent's entropy, subtract the weighted average entropy of its children (each child weighted by the fraction of points that landed in it). The bigger the drop, the better the split.

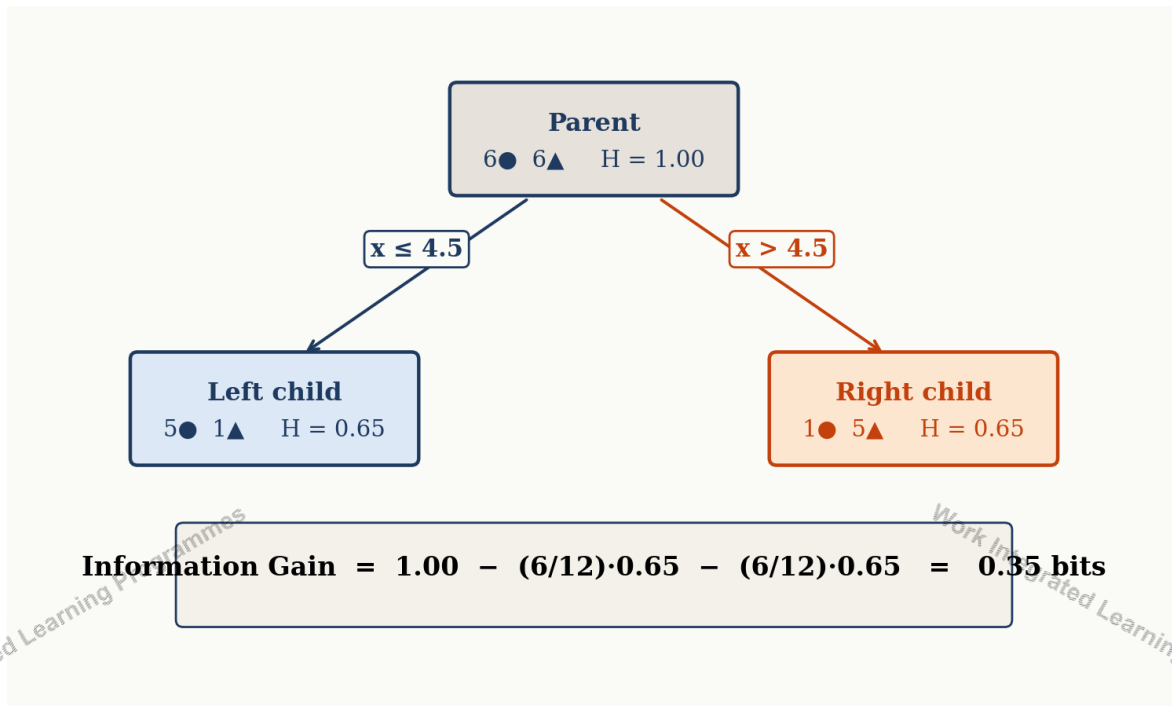


Figure 9.3 — Splitting a 50/50 parent into two ~83/17 children produces information gain ≈ 0.35 bits.

The full algorithm is now embarrassingly simple: at each node, try *every* feature and *every* possible split point, keep the split with the highest IG, and recurse on the children.

9.5 Worked example — information gain by hand

Imagine a tiny dataset of **14 emails** — 9 are spam, 5 are ham. We will compute the gain of splitting on the feature 'subject contains the word *FREE*'.

Step 1 — entropy of the parent (root)

$$p_{\text{spam}} = 9/14 = 0.643, \quad p_{\text{ham}} = 5/14 = 0.357$$

$$H(\text{root}) = -0.643 \cdot \log_2(0.643) - 0.357 \cdot \log_2(0.357) = \mathbf{0.940 \text{ bits}}$$

Step 2 — split data on the feature

Suppose, of the 14 emails:

- 8 contain 'FREE'** — and inside this subset, 7 are spam and 1 is ham. $\rightarrow p_{\text{spam}} = 7/8 = 0.875$, so $H = -0.875 \cdot \log_2(0.875) - 0.125 \cdot \log_2(0.125) = \mathbf{0.544 \text{ bits}}$
- 6 do not contain 'FREE'** — inside this subset, 2 are spam and 4 are ham. $\rightarrow p_{\text{spam}} = 2/6 = 0.333$, so $H = -0.333 \cdot \log_2(0.333) - 0.667 \cdot \log_2(0.667) = \mathbf{0.918 \text{ bits}}$

Step 3 — weighted child entropy

$$(8/14) \cdot 0.544 + (6/14) \cdot 0.918 = 0.311 + 0.393 = \mathbf{0.704 \text{ bits}}$$

Step 4 — information gain

$$IG = 0.940 - 0.704 = \mathbf{0.236 \text{ bits}}$$

What this number means. Knowing whether a subject contains the word 'FREE' gives us about 0.236 bits of information about whether the email is spam. If a competing feature, say 'subject is in ALL CAPS', yields $IG = 0.18$ bits, the tree prefers the 'FREE' question. That is the entire algorithm for choosing splits.

9.6 Construction algorithm (ID3 / CART)

GrowTree(node):

1. if node is pure or stop-condition reached → label it with majority class.

Return.

2. for every feature f and every split point s , compute $IG(f, s)$.
3. pick the (f^*, s^*) with the largest IG .
4. split data into two children — left = $\{x : x_{f^*} \leq s^*\}$, right = $\{x : x_{f^*} > s^*\}$.
5. recursively call $\text{GrowTree}(\text{left})$ and $\text{GrowTree}(\text{right})$.

Without stop conditions the tree will keep splitting until every leaf contains a single training point — that is perfect memorisation, also known as *overfitting*. The standard stops are:

- **max_depth** — never go below depth d .
- **min_samples_split** — refuse to split a node that has fewer than k points.
- **min_samples_leaf** — refuse a split if either child would have fewer than k points.
- **min_information_gain** — refuse a split if the gain is below a threshold ϵ .

Two pruning philosophies exist: **pre-pruning** uses the stop conditions above to keep the tree small as we build it; **post-pruning** grows the tree fully, then chops branches back using a held-out validation set.

9.7 Bias-Variance Tradeoff in Decision Trees

Decision trees are the cleanest illustration of the most important concept in machine learning. Tree depth is a single dial, and turning it changes everything.

Imagine you have a dataset and you train a tree of depth 1 — a *stump* with one split. It is too simple to capture the structure of the data. Train it on five different samples

of the data and you get five almost-identical stumps that are each roughly equally bad. **This is high bias.**

Now train a tree of depth 50 — every leaf has just one point. It memorises the training set perfectly. But train it on five different samples and you get five wildly different trees. **This is high variance.**

Somewhere in the middle is the sweet spot. The visualisation:

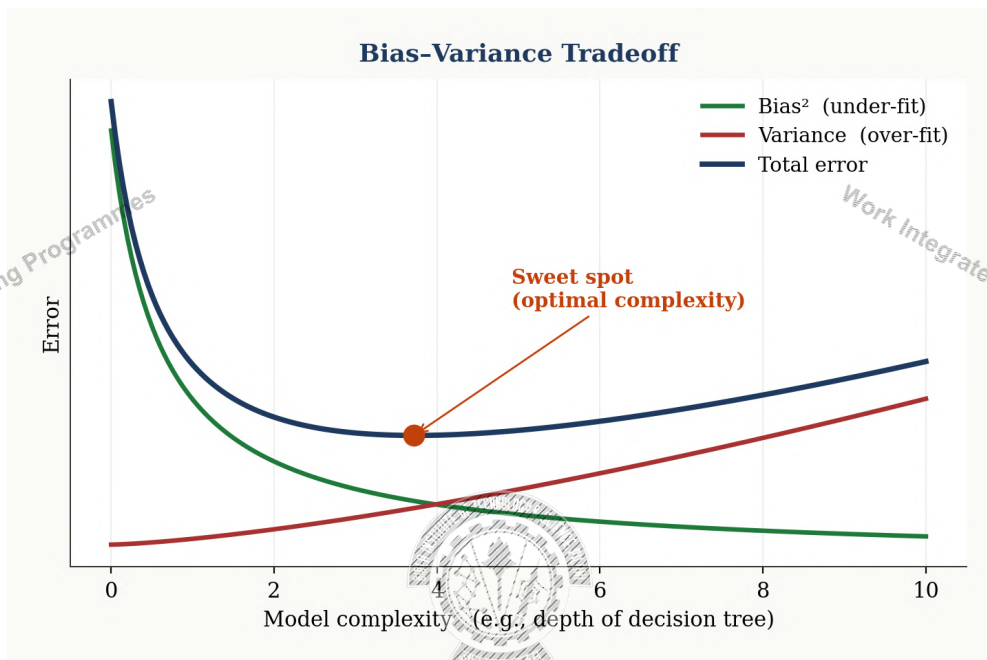


Figure 9.4 — Total expected error decomposes into bias squared, variance, and irreducible noise. The 'sweet spot' is where the U-curve bottoms out.

The decomposition formally

If we draw a fresh dataset, train a model on it and measure squared error on a fixed test point x , the expected error decomposes as:

$$E[(y - \hat{y}(x))^2] = (\text{Bias}[\hat{y}(x)])^2 + \text{Var}[\hat{y}(x)] + \sigma_{\text{noise}}^2$$

What each term *means*:

- **Bias** = how far the *average* prediction (averaged over many possible training datasets) is from the truth. Caused by an under-powered model. Linear models on curved data have high bias.
- **Variance** = how much the prediction *wobbles* when we re-train on a different dataset. Caused by an over-flexible model. Deep unpruned trees have high variance.
- **Noise** = irreducible. Set by the labellers, the sensor, the world. No model can remove it.

Practical recipe for trees. Start deep (let the model overfit a bit). Watch the validation accuracy. Pull back depth (or increase `min_samples_leaf`) until the validation curve *stops* dropping. Yes, the training accuracy will go down a little — that is exactly what we want. We are trading bias for less variance.

Why does ensembling fix this? **If you average many high-variance trees, the variance shrinks while the bias stays the same.** That is the secret behind Random Forests — see Session 10.

9.8 When to use, and when not to

Strengths	Weaknesses
Easy to interpret — you can print the tree and read it like a flowchart.	A single tree is unstable: change one training point, the tree re-arranges itself.
Handles numeric and categorical features natively. No scaling needed.	Cannot model smooth functions. Boundaries are stair-step (axis-aligned).
Captures non-linear interactions automatically — no feature engineering.	Greedy: never reconsiders an early split if a later one would have been better.
Robust to irrelevant features and outliers.	Without pruning, very prone to overfitting.

Use a single tree when interpretability is the absolute priority — for example, a regulator wants to audit your loan-approval logic. **Use a tree ensemble when** accuracy matters more than explanation — and that is most of the time in practice.

9.9 Common pitfalls

- **Forgetting to set `max_depth`.** An unconstrained tree on noisy data will overfit perfectly. Always start with a depth in the 5-10 range and grow if validation supports it.
- **Reading single-tree feature importance as causal.** If two features are correlated, a tree may use one and ignore the other; that does not mean the second one is unimportant.
- **Splitting on an ID column.** Yes, this happens. An ID gives 0 entropy at every leaf and is the 'best' split — but it generalises to nothing. Drop ID columns before training.
- **Ignoring class imbalance.** If 99% of your data is one class, a tree may produce a one-leaf 'tree' that always predicts that class. Use class weights or balanced sampling.

End-of-session takeaway. Decision trees teach three lessons we will use throughout this guide:

1. **Information theory gives us a principled way to score splits** — entropy and information gain.
2. **Bias-variance is real and visible** — turn the depth dial and watch the test error.
3. **The variance of a single model can be tamed by averaging many of them** — which is exactly what Session 10 is about.



Work Integrated Learning Programmes



Work Integrated Learning Programmes

Work Integrated Learning Programmes



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes

CONTACT SESSION 10

Ensembles — Bagging, Boosting, Random Forest

Ensembles — When Many Average Models Beat One Genius

10.1 The motivation, in plain English

We saw at the end of Session 9 that a single deep decision tree has *low bias but very high variance*. Train it on a slightly different sample and it produces a wildly different tree. That instability is the whole problem.

What if, instead of trusting a single tree, we trained *many* trees — each on a slightly different version of the data — and asked them to vote? When some trees over-react one way and others over-react the other, their errors might *cancel out*, leaving us with something more robust than any single tree.

That single intuition powers an entire family of algorithms called **ensemble methods**. They sit at the heart of modern classical machine learning — XGBoost, LightGBM, CatBoost, Random Forest, AdaBoost — and routinely win Kaggle competitions and real-world deployments. Understanding ensembles is non-negotiable.

“The wisdom of the crowd, when each member is even mildly competent and their mistakes are roughly uncorrelated, beats any individual expert.”

10.2 Why averaging actually reduces variance

Consider M independent estimators, each with the same bias and variance σ^2 . If we average their predictions:

$$\text{Var}((1/M) \cdot \sum \hat{y}_i) = \sigma^2 / M$$

The variance shrinks linearly with the number of estimators. The bias is unchanged. We pay nothing in bias and gain a lot in variance — that is the holy grail.

But in real life the estimators are *not* independent: trees trained on similar data give similar predictions. If their correlation is ρ , the variance becomes:

$$\text{Var}(\text{ensemble}) = \rho \cdot \sigma^2 + (1 - \rho) \cdot \sigma^2 / M$$



Read this carefully. The first term, $\rho \cdot \sigma^2$, is a *floor* — it does not vanish even with infinitely many trees. So the trick to a strong ensemble is not just adding more trees; it is making them **diverse**. Bootstrapping, feature subsampling, and reweighting are different strategies to make trees disagree in productive ways.



Work Integrated Learning Programmes

10.3 Bagging — Bootstrap Aggregating

Bagging — short for *Bootstrap Aggregating* — is the simplest, most elegant variance-reduction technique. Three lines:

- 1 Take many **random samples with replacement** from your training data — these are 'bootstrap samples'.
- 2 Train one model (typically a deep, unpruned decision tree) on each sample.
- 3 To predict for a new point, ask all models and combine: **majority vote** for classification, **average** for regression.

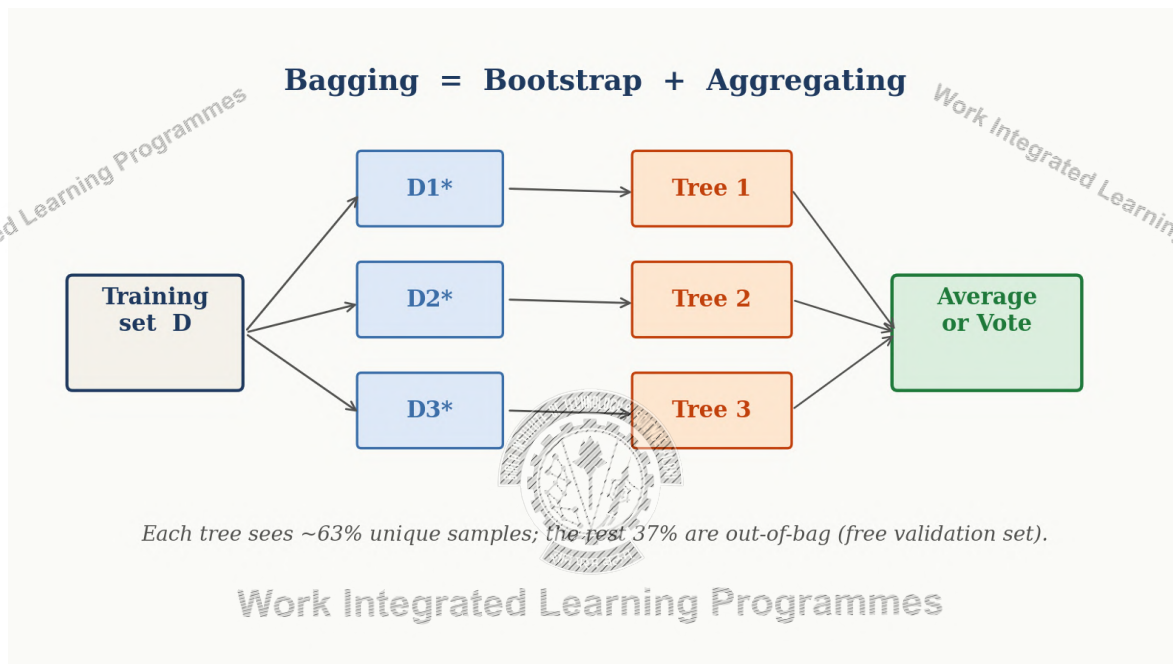


Figure 10.1 — The bagging pipeline: bootstrap → train independent models → aggregate.

Why bootstrap sampling? The 63% rule

If we draw n samples with replacement from a dataset of size n , what fraction of *unique* original points appear in the bootstrap sample?

$$P(\text{point } i \text{ is NEVER picked}) = (1 - 1/n)^n \rightarrow 1/e \approx 0.368 \text{ as } n \text{ grows.}$$

So roughly **63.2%** of unique points appear in each bootstrap sample, while **36.8%** are missing. The missing ones are called **out-of-bag (OOB) samples**. Use them to validate each tree on data that tree never saw — average across trees and you get a free estimate of generalisation error, without ever needing a held-out validation set.

Worked numbers — variance reduction

Suppose a single deep tree on our problem has training-set variance $\sigma^2 = 0.10$. If our 100 bagged trees have correlation $\rho = 0.50$:

$$\text{Var}(\text{bagged}) = 0.50 \cdot 0.10 + (1 - 0.50) \cdot 0.10 / 100 = 0.0500 + 0.0005 = 0.0505$$

We dropped variance from 0.10 down to 0.05 — half — at the cost of training 100 trees. If we could push correlation down to 0.10:

$$\text{Var}(\text{bagged}) = 0.10 \cdot 0.10 + 0.90 \cdot 0.10 / 100 = 0.010 + 0.0009 = 0.0109$$

Almost a 10× reduction. **Driving down correlation matters more than adding more trees.** Hold this thought — Random Forest is going to use exactly this lever.



Work Integrated Learning Programmes

10.4 Boosting — fix the previous model's mistakes

Boosting takes the opposite philosophy from bagging. Instead of training models *independently* and averaging, boosting trains models *sequentially* — each new model concentrates specifically on the examples the previous models got wrong.

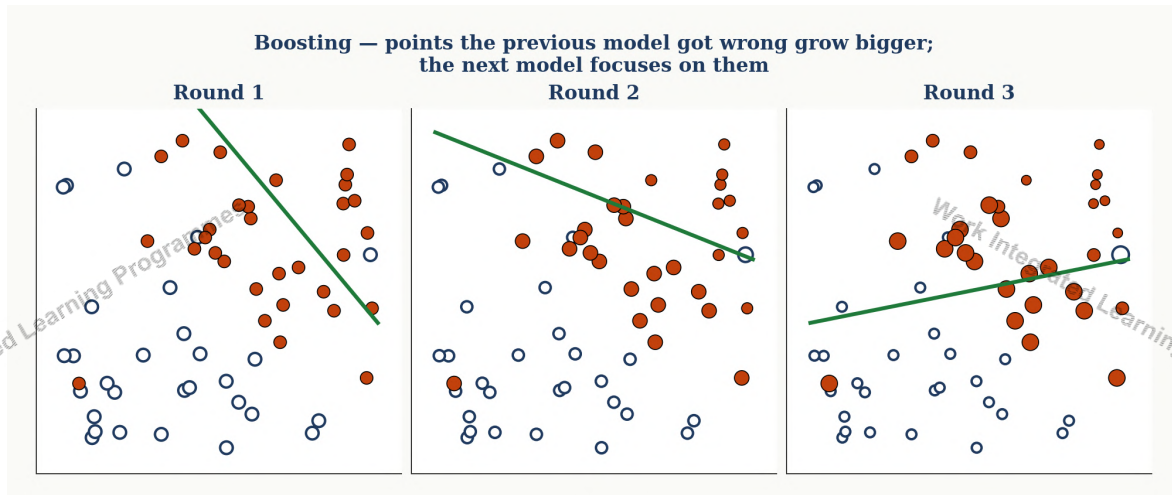


Figure 10.2 — Three rounds of boosting. After each round, points the previous model misclassified grow visibly larger so the next model focuses on them.

The tutor analogy

Imagine a student preparing for an exam. The first tutor teaches everything generally. After a practice test, we identify topics the student got wrong. The second tutor focuses *only* on those weak topics. After another test, the third tutor focuses on what is still wrong. After many rounds the cracks have all been filled. That is boosting — every tutor specialises in fixing the previous tutor's leftovers.

AdaBoost — the original algorithm in detail

AdaBoost (Adaptive Boosting) for binary classification with labels $y \in \{-1, +1\}$:

1. Initialise weights $w_i = 1 / n$ for every training point i .
2. For round $t = 1 \dots T$:
 - (a) Train a weak learner h_t on the weighted data.
 - (b) Compute weighted error $\varepsilon_t = \sum_i w_i \cdot 1[y_i \neq h_t(x_i)]$.
 - (c) Compute voting weight $\alpha_t = \frac{1}{2} \cdot \ln((1 - \varepsilon_t) / \varepsilon_t)$.
 - (d) Update weights $w_i \leftarrow w_i \cdot \exp(-\alpha_t \cdot y_i \cdot h_t(x_i))$, then normalise so they sum to 1.
3. Final classifier: $H(x) = \text{sign}(\sum_t \alpha_t \cdot h_t(x))$.

The voting weight α_t is the cleverest piece. A learner with $\varepsilon = 0.5$ (random guessing) gets $\alpha = 0$ — it is given no voice. A learner with $\varepsilon = 0$ (perfect on the weighted data) gets $\alpha = \infty$ — it dominates. A learner with $\varepsilon < 0.5$ gets a positive vote; $\varepsilon > 0.5$ a negative vote (its predictions are flipped).

Worked AdaBoost step

Round 1 produces a weak learner h_1 with error $\varepsilon_1 = 0.20$. Then:

$$\alpha_1 = \frac{1}{2} \cdot \ln(0.80 / 0.20) = \frac{1}{2} \cdot \ln(4) \approx \mathbf{0.693}$$

Now the weight update for each training point:

- **Misclassified** points satisfy $y_i \cdot h_1(x_i) = -1$ — their weight is multiplied by $\exp(0.693) \approx \mathbf{2.0}$ (their voice doubles).
- **Correctly classified** points satisfy $y_i \cdot h_1(x_i) = +1$ — their weight is multiplied by $\exp(-0.693) \approx \mathbf{0.5}$ (their voice halves).

After normalisation, the misclassified points are now relatively much louder. The next weak learner h_2 will pay close attention to them. Repeat T times and the model becomes a sum of weak learners, each one filling the previous round's cracks.

Modern boosting — gradient boosting and XGBoost

AdaBoost is the original. Modern boosting libraries — **Gradient Boosting Machines, XGBoost, LightGBM, CatBoost** — generalise the idea: at each round, fit the weak learner to the *negative gradient* of the loss with respect to the previous model's predictions. For squared error this turns into 'fit on the residuals' — and exactly that simple line of code wins half of all tabular-data Kaggle competitions.

10.5 Bagging vs Boosting at a glance

Aspect	Bagging	Boosting
How models are trained	In parallel — each is independent	Sequentially — each fixes the previous
Sample weighting	Uniform (random bootstrap)	Re-weighted; mistakes get heavier
Reduces mainly	Variance	Bias (and variance, when tuned)
Sensitivity to noise	Low — noisy points are randomised away	High — boosting can chase outliers
Typical weak learner	Deep, unpruned tree	Shallow tree (depth 1–6)
Parallelisation	Embarrassingly parallel	Mostly sequential
Typical libraries	RandomForest (sklearn), Bagging-of-anything	XGBoost, LightGBM, CatBoost, AdaBoost



Work Integrated Learning Programmes

10.6 Random Forest — bagging on steroids

Random Forest is bagging applied to decision trees, with one extra trick that turns out to be transformative. Recall the variance formula:

$$\text{Var}(\text{ensemble}) = \rho \cdot \sigma^2 + (1 - \rho) \cdot \sigma^2 / M$$

The variance is dominated by ρ . To beat down ρ , Random Forest forces the trees to be different from each other in a clever way: at every split, instead of considering all d features, it picks the best split among only a **random subset of features** (typically $\sqrt{d}$ for classification, $d/3$ for regression).

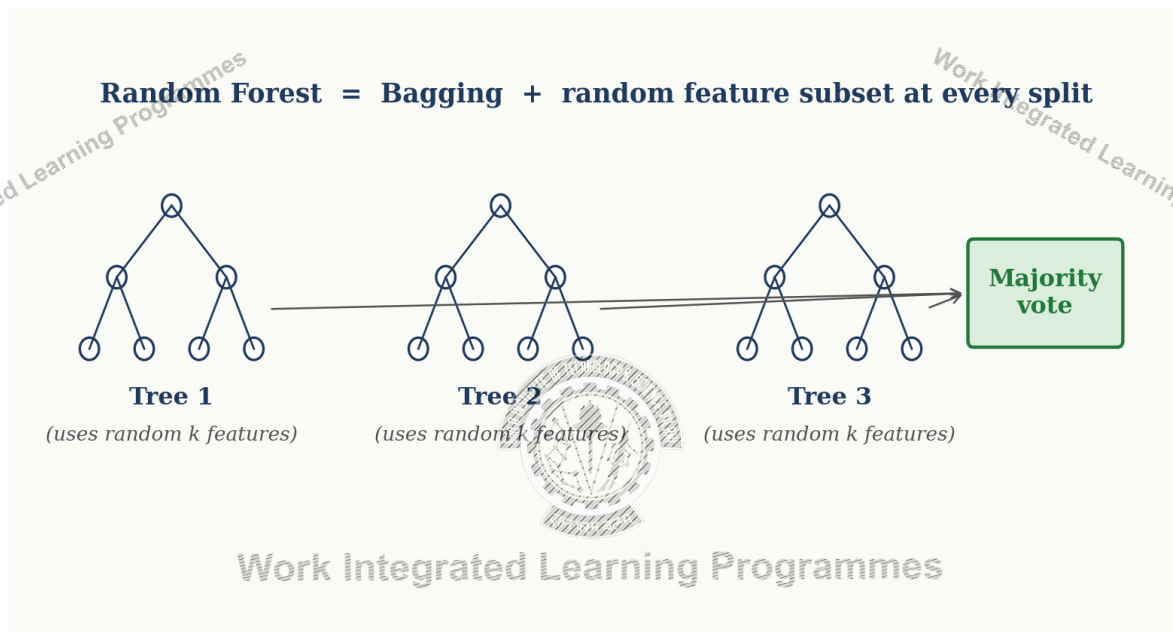


Figure 10.3 — Random Forest combines two sources of randomness: each tree sees a bootstrap of rows AND a random subset of features at every split.

Why this works — the doctor analogy

Imagine you ask 100 doctors to diagnose a patient. If they all trained at the same hospital, saw the same patients, and were allowed to look at the same symptoms, they will agree on the same misdiagnosis. But give each doctor a different sample of patients *and* only allow them to look at a random subset of symptoms, and their mistakes become independent. A majority vote among 100 such doctors is remarkably hard to fool. That is the whole intuition behind Random Forest.

Hyperparameters that actually matter

- **n_estimators** — number of trees. More is always at least as good (variance only goes down). Stop when the validation curve plateaus. Typical: 100 – 1000.
- **max_features** — features considered at each split. Smaller → more diverse trees → lower correlation → lower ensemble variance. Defaults: $\sqrt{d}$ for classification, $d/3$ for regression.

- **max_depth, min_samples_leaf** — controls each tree's complexity. Forests are robust enough that you can usually leave trees deep and still get good results.
- **n_jobs** — random forest is embarrassingly parallel. Set to -1 to use every core.

Feature importance — a free bonus

Every tree records, at each split, the impurity drop. Aggregate this across all trees, and you get a per-feature importance score. This is one of the few interpretability tools that comes *built in* with a black-box model. Be careful, though — feature importance is biased toward high-cardinality features. Use **permutation importance** (shuffle a feature, see how much accuracy drops) for a more honest reading.

10.7 When to use which ensemble

- **Random Forest** — your default. Robust, no tuning required, fast to train, parallel.
- **Gradient Boosting (XGBoost / LightGBM)** — when you want the absolute best score and have time to tune.
- **AdaBoost** — when you have a clean dataset and a small set of strong-but-imperfect features.
- **Bagging anything** — bagging works on any base learner with high variance: even neural networks (called 'snapshot ensembles' in that context).

End-of-session takeaway. Variance reduction is the ML equivalent of insurance — averaging many imperfect models gives you a model that is hard to embarrass on any single test point. Bagging plays it safe; boosting plays it sharp; Random Forest splits the difference and wins.

10.8 Choosing between forests and neural networks

Students often ask: *why use a Random Forest at all when neural networks exist?* The honest answer is that on a wide swath of real-world problems — particularly modest-sized **tabular** data — Random Forest still beats deep networks comfortably. Three reasons:

- **Sample efficiency.** Trees handle small datasets natively. With 50k rows and 30 mixed-type features, a deep neural network has the capacity to memorise rather than learn. A forest just averages many already-decent trees and does well.

- **No structural prior to exploit.** Deep networks shine when inputs have spatial structure (images), sequential structure (text), or hierarchical patterns. Tabular data has none of those — there is nothing for depth to discover.
- **Robustness with no tuning.** Random Forests are nearly hyperparameter-free; deep networks demand careful regularisation, learning rates and architectures.

Conversely, neural networks pull ahead when you have **lots of data** (millions of examples) and **structured inputs** (images, audio, text, sequences). The rule of thumb: pick the model whose inductive bias matches the structure of your problem.



Work Integrated Learning Programmes

CONTACT SESSION 11

Unsupervised Learning — PCA, K-Means, DBSCAN

Unsupervised Learning — Finding Structure Without Labels

11.1 Why unsupervised learning matters

Most data in the world has no labels. The photos on your phone, the transactions on a credit card, customer behaviour, gene expression, raw text from the web — none of it comes with a 'correct answer' attached. Asking what we can still learn from such data is the question of unsupervised learning. The answers come in two flavours:

- **Dimensionality reduction** — compress each high-dimensional point into a small number of coordinates while preserving most of the structure. PCA is the classic.
- **Clustering** — group similar points together so we can label or treat them collectively. K-Means and DBSCAN are the workhorses.

Unsupervised learning shows up in pre-processing pipelines (PCA before regression), in customer-segmentation analyses, in anomaly detection, in recommendation systems, in pre-training of modern language models, and in countless other places. Master the three classical algorithms below and you will have a strong base.

11.2 Principal Component Analysis (PCA)

One sentence. PCA rotates your coordinate system so the new axes point along the directions of greatest variance in your data. You can then keep just the first few axes and discard the rest, losing very little information.

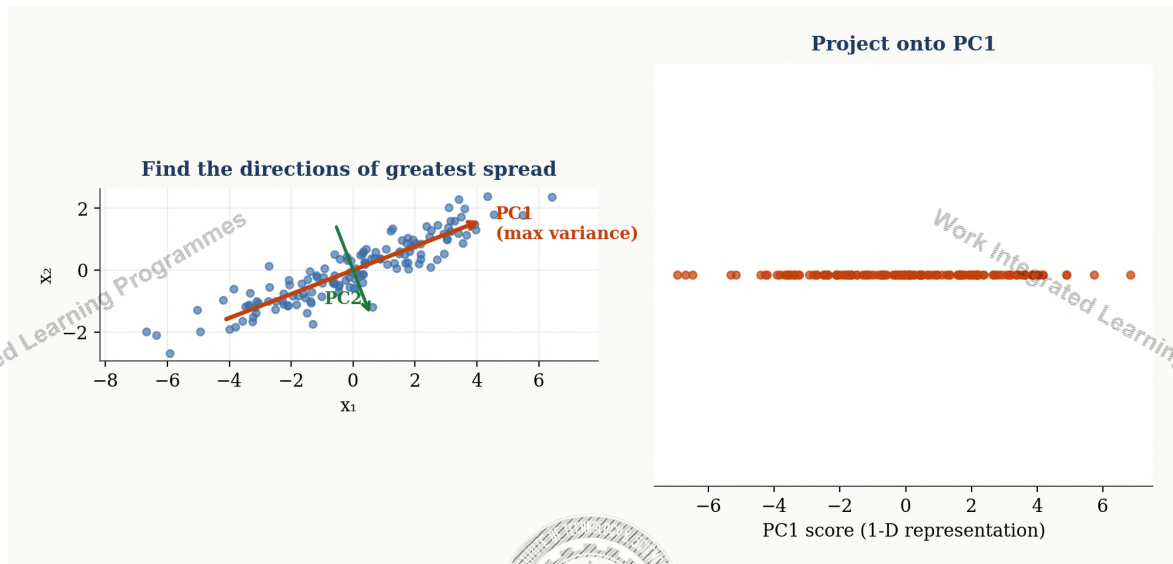


Figure 11.1 — Left: raw 2-D data with PC1 (max variance) and PC2 (perpendicular). Right: each point projected onto PC1 — a 1-D representation that preserves most of the variance.

Geometric intuition — three pictures in one

Imagine a 2-D cloud of points roughly aligned along a diagonal. Most of the variation is along that diagonal; the perpendicular direction shows little variation. PCA does three things at once:

- **It rotates the coordinate system** so the new x-axis (called PC1) points along the diagonal, and the new y-axis (PC2) is perpendicular.
- **It ranks the new axes by variance** — PC1 explains the most variance, PC2 the next-most, and so on.
- **It discards small-variance axes** to compress: throw away PC2 and you keep ~80% of the information using only one number per point.

The math, in four short steps

Step 1 — Centre the data (mean removal):

$$X_{\text{centred}} = X - \text{mean}(X)$$

Step 2 — Compute the covariance matrix:

$$C = (1/n) \cdot X_{\text{centred}}^T \cdot X_{\text{centred}} \quad (\text{a } d \times d \text{ matrix})$$

C_{ij} tells you how feature i and feature j vary together. The diagonal entries are individual variances, the off-diagonal entries are covariances.

Step 3 — Eigendecompose C:

$$C \cdot v_k = \lambda_k \cdot v_k$$

The eigenvectors v_k are the principal directions; the eigenvalues λ_k are the variances along those directions. Sort them from largest to smallest.

Step 4 — Project:

$$z = V_k^T \cdot x \quad (V_k = \text{top-}k \text{ eigenvectors stacked as columns})$$

How many components to keep?

The fraction of variance explained by the first k components is:

$$\text{ExplainedRatio}(k) = (\lambda_1 + \dots + \lambda_k) / (\lambda_1 + \dots + \lambda_d)$$

Plot this — the 'scree plot' — and pick a k that captures, say, 95% of the variance, or pick the elbow where adding more components barely helps. There is no magic threshold; the right k depends on what you'll do downstream.

Two cautionary extremes

- **Keeping too few components** (e.g., only the top 1 or 2 because they 'look dominant') discards useful signal. Just because PC1 explains 22% of variance doesn't mean the remaining 78% is noise — the rest can carry meaningful structure spread thinly across many components.
- **Mechanically demanding 95% variance** can also be wrong. On a high-dimensional dataset where variance is spread thinly across many features (e.g., 80 customer features with no single dominant pattern), 95% may need 60+ components — almost no compression. At that point PCA is barely doing anything; consider whether the right tool is a non-linear method (autoencoders, UMAP) or whether the data simply doesn't have a low-dimensional structure to exploit.

Practical recipe. Plot the cumulative explained-variance curve. Pick the smallest k that satisfies *both*: (a) captures a reasonable fraction of variance (often 80 - 95% depending on the downstream task), and (b) sits at or just beyond a visible elbow where the curve flattens. Cross-validate downstream performance for a small range of k around that point — let the actual task decide.

Worked example — a tiny 2-D dataset

Six points: (2, 0), (0, 2), (3, 1), (1, 3), (4, 2), (2, 4). Mean is (2, 2). After centring, the covariance matrix becomes:

$$C = \begin{bmatrix} 2.00 & 1.20 \\ 1.20 & 2.00 \end{bmatrix}$$

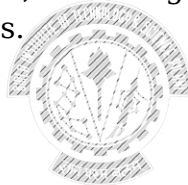
Eigenvalues are $\lambda_1 = 3.20$ and $\lambda_2 = 0.80$ (trace = 4, det = 2.56). The top eigenvector is $(1, 1) / \sqrt{2}$ — the data lies mainly along the diagonal. Projecting onto it explains:

$$\text{ExplainedRatio}(1) = 3.20 / (3.20 + 0.80) = \mathbf{0.80} \rightarrow 80\% \text{ of variance}$$

We have replaced two numbers per point with one, while keeping 80% of the structure. On real datasets like images, where d can be in the tens of thousands, the compression is far more dramatic.

Practical tips

- **Always standardise features first** — otherwise a feature measured in millimetres will dominate one measured in metres.
- **PCA is linear.** If your data lies on a curved manifold (like a Swiss roll), use t-SNE or UMAP instead.
- **The principal components are not interpretable as features.** They are linear combinations of the originals, often with mixed signs.
- **PCA can be used for compression, denoising, or as a pre-processing step.** When training is slow, reducing 768 features to 50 can give 10× speed-ups with negligible loss.



Work Integrated Learning Programmes

11.3 K-Means Clustering

One sentence. K-Means partitions n points into K groups so that each point belongs to the cluster whose centroid is closest, and centroids are themselves the means of their groups.

K-Means: assign each point to the nearest centroid → recompute each centroid as the mean of its members → repeat

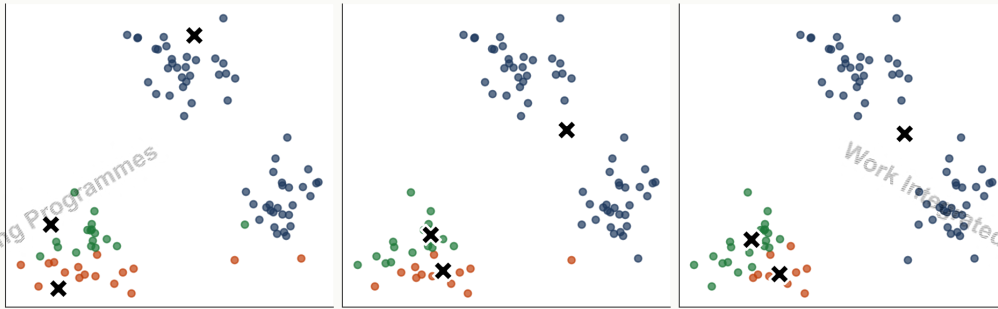


Figure 11.2 — Three iterations of K-Means. Assign each point to the nearest centroid (colours), then recompute each centroid as the mean of its members. Repeat until centroids stop moving.

The objective and algorithm

K-Means minimises the within-cluster sum of squares (WCSS), also called inertia:

$$J = \sum_{k=1..K} \sum_{x \in C_k} \|x - \mu_k\|^2$$

Lloyd's algorithm alternates two steps until convergence:

- 1 **Assignment step.** Each point joins the cluster of its nearest centroid.
- 2 **Update step.** Each centroid becomes the mean of its assigned points.

Two facts that surprise students: (a) the WCSS objective is *guaranteed* to decrease at every step (or stay the same), and (b) convergence is reached in a finite number of steps. The catch: the algorithm can get stuck in a local minimum. Run it with several random initialisations and keep the best.

Choosing K — elbow and silhouette

Elbow method. Plot J versus K . Pick the K where adding a new cluster gives diminishing returns — the curve bends. Subjective but often clear.

A trap students fall into. WCSS is *monotonically decreasing* in K — every additional cluster fits the data a little more tightly. In fact, J reaches zero when $K = n$ (one cluster per point). So picking the K with the *lowest* WCSS is meaningless: you would always pick the largest K you tried. The elbow is the answer because it captures the point where extra clusters stop reducing error meaningfully — that is where additional segments deliver diminishing business value.

Silhouette score. For each point i , compute:

$$s(i) = (b(i) - a(i)) / \max(a(i), b(i))$$

where $a(i)$ is the mean distance from i to other points in its own cluster, and $b(i)$ is the mean distance from i to points in the next-closest cluster. The score ranges from -1 (terrible) to $+1$ (perfect). Average it across all points and pick the K that maximises the average silhouette.

K-Means++ initialisation

Random initial centroids can all land near each other and produce a terrible local optimum. **K-Means++** picks the first centroid uniformly at random, then each subsequent centroid with probability *proportional to the squared distance* to the nearest already-chosen centroid. The centroids start spread out, convergence is faster, and final WCSS is provably within a logarithmic factor of optimal. Modern libraries default to K-Means++; use it.

Limitations of K-Means

- Assumes clusters are **spherical and roughly equal-sized**. Elongated or curved clusters confuse it.
- Sensitive to feature scale** — always standardise first.
- Requires K to be specified upfront — try the elbow / silhouette methods.
- Sensitive to outliers** — a single faraway point can pull a centroid wildly off.
- Can find different local optima — run with several random seeds.

11.4 DBSCAN — Density-Based Clustering

One sentence. DBSCAN labels regions of high density as clusters, regardless of shape, and isolated points as noise — without ever asking you for K.

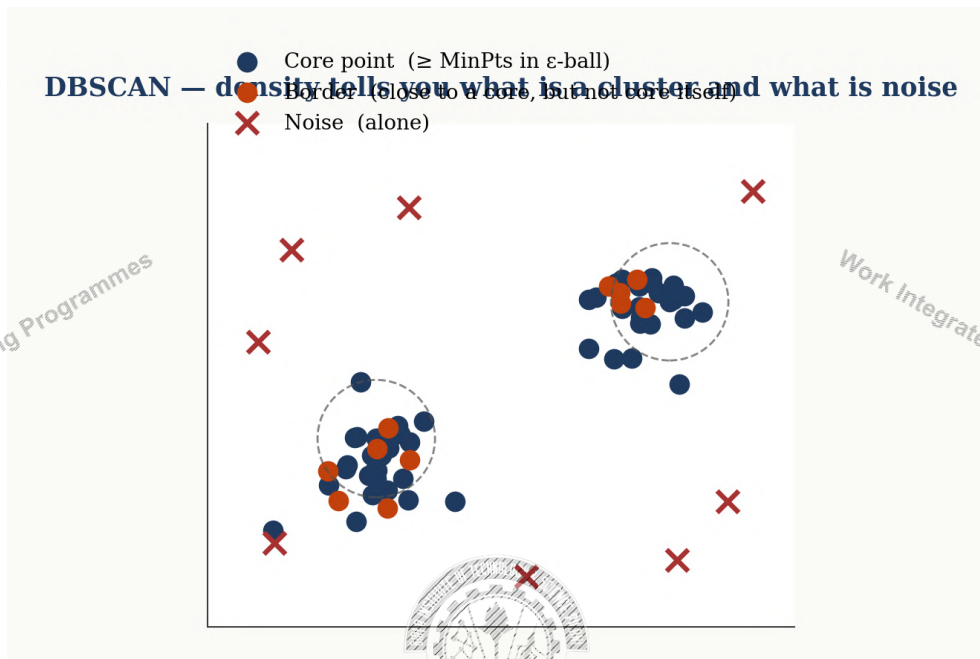


Figure 11.3 — DBSCAN partitions points into three categories: core (dense), border (close to a core), and noise (alone).

Two hyperparameters — and that is it

- **ϵ (epsilon)** — the radius of the neighbourhood we examine around each point.
- **MinPts** — the minimum number of points (including itself) inside an ϵ -ball needed to call a point a 'core' point.

Three kinds of points emerge naturally:

- **Core point** — has at least MinPts neighbours within distance ϵ . It is the seed of a cluster.
- **Border point** — within distance ϵ of a core point but with fewer than MinPts neighbours itself.
- **Noise point** — neither core nor border. The algorithm explicitly labels it as noise (label = -1 in scikit-learn).

The algorithm in pseudocode

```

DBSCAN( $\epsilon$ , MinPts):
  for each unvisited point  $p$  in the dataset:
    mark  $p$  as visited;
     $N = \epsilon$ -neighbourhood( $p$ );
    if  $|N| < \text{MinPts}$ :
      label  $p$  as noise (it can later become a border point);
    else:
      start a new cluster; expand it by recursively adding all density-reachable
      points.
  
```

Choosing ϵ — the k-distance plot

Too small ϵ and everything becomes noise. Too large ϵ and everything is one big cluster. The trick: pick a value of MinPts (rule of thumb: $2 \cdot d$ for a d -dimensional dataset), then for every point compute the distance to its MinPts-th nearest neighbour. Sort those distances and plot — there is usually a clear elbow. The ϵ at the elbow is the right ϵ .

11.5 K-Means vs DBSCAN — when to choose which

Question	K-Means	DBSCAN
Need to specify K?	Yes	No (auto)
Cluster shape?	Spherical only	Arbitrary
Outlier handling?	Forces every point into a cluster	Labels noise explicitly
Hyperparameters	K and a random seed	ϵ and MinPts
Scales to big n ?	Yes (linear)	Worse — $O(n \log n)$ with index, $O(n^2)$ without
Cluster size assumption?	Roughly equal	No assumption
Reproducible?	Depends on seed	Deterministic given parameters



End-of-session takeaway. Unsupervised learning is the single most under-used family of techniques in industry — partly because the metrics are subjective. Use PCA every time you have more than ~ 30 features. Use K-Means when you have a rough guess of K and roughly spherical clusters. Use DBSCAN when clusters are weirdly shaped or when noise points are themselves what you care about (anomaly detection).



Work Integrated Learning Programmes

CONTACT SESSION 12

Artificial Neural Networks

Artificial Neural Networks — From One Neuron to a Network

12.1 Why neural networks

Almost every modern AI achievement — image recognition, speech recognition, machine translation, protein folding, code generation, even chess and Go — is powered by neural networks. They are not magical. Underneath all the engineering complexity, a neural network is just stacked linear regressions glued together by non-linearities. That is the idea we will build, brick by brick.

12.2 The Single Neuron — Perceptron

One sentence. A neuron computes a weighted sum of its inputs, adds a bias, and squashes the result through a non-linear activation function.

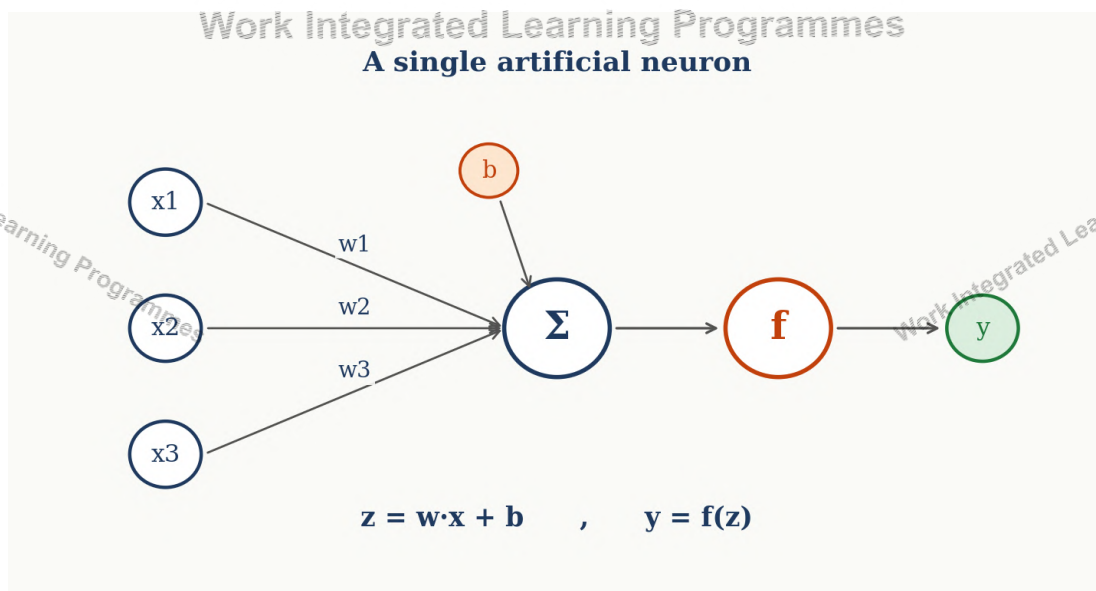


Figure 12.1 — A single artificial neuron. Inputs are weighted, summed, biased, and passed through f .

Mathematically, for inputs $x = (x_1, \dots, x_d)$:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_d \cdot x_d + b$$

$$y = f(z)$$

Note that without f — the activation function — stacking many neurons would just give us another linear function. The non-linearity is the magic ingredient that lets networks model curves, surfaces and shapes of arbitrary complexity.

Boolean logic with one neuron

A single neuron with a step activation can compute AND, OR, NOT — but **not** XOR. AND with two inputs:

AND : $w_1 = 1, w_2 = 1, b = -1.5, f = \text{step}$. Then $z > 0$ only when both $x_1 = x_2 = 1$.

XOR cannot be expressed by any single linear separator (this is the famous Minsky-Papert result that nearly killed the field in 1969). Solving XOR requires **two layers** — that is the next step.



Work Integrated Learning Programmes

12.3 The Multi-Layer Perceptron (MLP)

Stack many neurons in a layer, then stack many layers. Each layer takes the previous layer's output, applies its own (W, b, σ) , and passes the result on. That is an MLP.

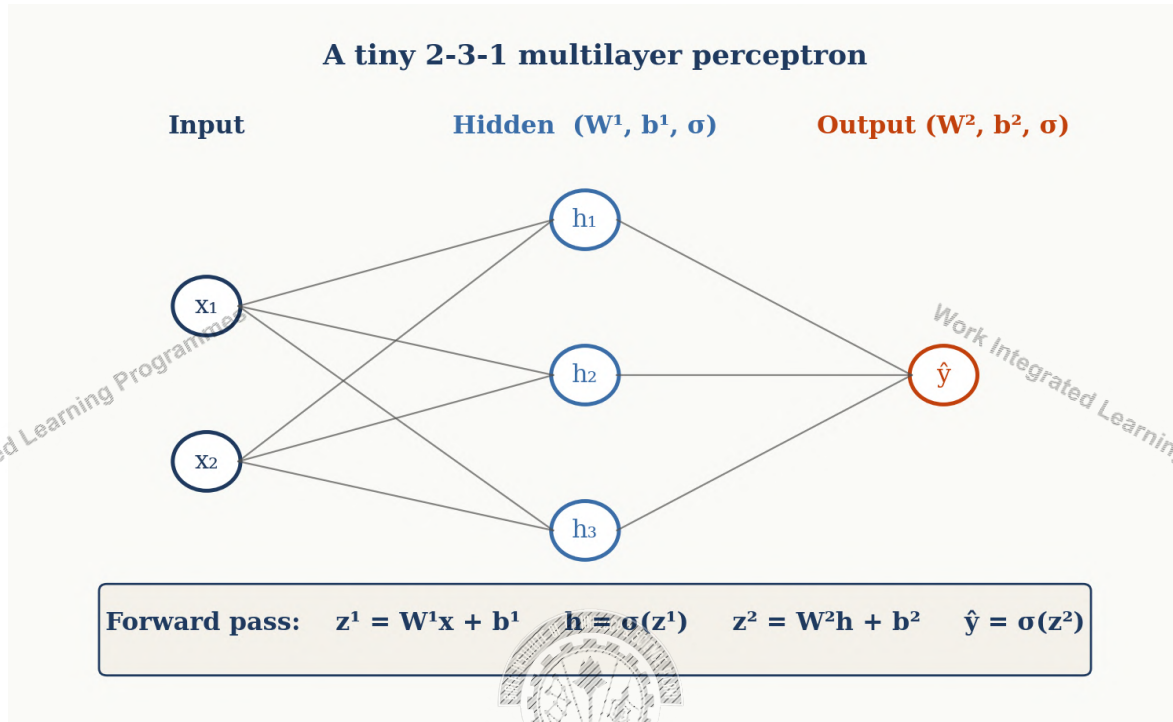


Figure 12.2 — A tiny 2-3-1 MLP. Two inputs, one hidden layer of three neurons, one output. Forward pass shown at the bottom.

12.4 Forward Pass — worked by hand on Figure 12.2

Let us forward-propagate one input through the 2-3-1 network of Figure 12.2 with concrete numbers. We will use the sigmoid $\sigma(z) = 1 / (1 + e^{-z})$ for both layers (in real life you would use ReLU; sigmoid keeps the example easy to read).

Inputs and parameters

$$x = [1.0, 2.0]^T$$

$$W^1 = \begin{bmatrix} 0.5 & -0.2 \\ 0.1 & 0.4 \\ -0.3 & 0.6 \end{bmatrix}, \quad b^1 = [0.0, 0.1, -0.2]$$

$$W^2 = [0.7, -0.5, 0.2], \quad b^2 = 0.05$$

Step 1 — hidden pre-activations

$$z^1_1 = 0.5 \cdot 1 + (-0.2) \cdot 2 + 0.0 = \mathbf{0.10}$$

$$z^1_2 = 0.1 \cdot 1 + 0.4 \cdot 2 + 0.1 = \mathbf{1.00}$$

$$z^1_3 = (-0.3) \cdot 1 + 0.6 \cdot 2 - 0.2 = \mathbf{0.70}$$

Step 2 — hidden activations (sigmoid)

$$h_1 = \sigma(0.10) = \mathbf{0.5250}$$

$$h_2 = \sigma(1.00) = \mathbf{0.7311}$$

$$h_3 = \sigma(0.70) = \mathbf{0.6682}$$

Step 3 — output pre-activation

$$\begin{aligned} z^2 &= 0.7 \cdot 0.5250 + (-0.5) \cdot 0.7311 + 0.2 \cdot 0.6682 + 0.05 \\ &= 0.3675 - 0.3656 + 0.1336 + 0.05 = \mathbf{0.1856} \end{aligned}$$

Step 4 — output activation

$$\hat{y} = \sigma(0.1856) = \mathbf{0.5463}$$

What we just did. In one tiny calculation we performed everything a neural network does at inference time. The network predicts the positive class ($\hat{y} \geq 0.5$). Training the network is the inverse: *given* many examples, find the W's and b's that make $\hat{y}$ match the true y. That is the job of gradient descent and backpropagation.



Work Integrated Learning Programmes

12.5 Loss functions and gradient descent

We measure how wrong the model is using a **loss function**. The two you will use most:

Squared error (regression): $L = \frac{1}{2} \cdot (y - \hat{y})^2$

Binary cross-entropy (classification): $L = -y \cdot \log(\hat{y}) - (1 - y) \cdot \log(1 - \hat{y})$

We update every parameter in the direction that reduces L the most:

$$\theta \leftarrow \theta - \eta \cdot \partial L / \partial \theta$$

η is the **learning rate**. Too small and learning crawls; too big and the optimisation bounces around the minimum. Typical values: $1e-3$ to $1e-1$, often with adaptive optimisers like Adam.

12.6 Backpropagation in plain English

Computing $\partial L / \partial \theta$ for every weight in a deep network looks impossible until you remember the chain rule from calculus. Backpropagation applies the chain rule *recursively* from the output backwards. For our network, with sigmoid activations and binary cross-entropy:

$\delta^2 = \hat{y} - y \leftarrow$ the output error (a happy property of sigmoid + BCE)

$$\partial L / \partial W^2 = \delta^2 \cdot h^T$$

$$\delta^1 = (W^{2T} \cdot \delta^2) \odot \sigma'(z^1)$$

$$\partial L / \partial W^1 = \delta^1 \cdot x^T$$

Here $\odot$ is element-wise product. In English: errors flow *backwards* through the network, and at each layer they get multiplied by the local Jacobian — the activation's derivative. The gradient w.r.t. that layer's weights is then just an outer product of the back-flowing error with the layer's input.

The intuition you must keep. Forward pass = make a prediction. Backward pass = propagate the error backwards. The chain rule lets us compute every weight's contribution to the loss without ever doing anything more complicated than matrix multiplications and element-wise function applications. That is the single algorithmic idea that powers all of modern deep learning.

12.7 Why Networks Can Learn (Almost) Anything

Universal Approximation Theorem (informal). An MLP with a single hidden layer of sufficient width can approximate any continuous function on a bounded domain to arbitrary accuracy.

Caveat: 'sufficient' might mean exponentially many neurons. In practice, depth (many layers) is what makes networks *efficient* — deep networks can represent the same function with exponentially fewer neurons than wide-but-shallow ones.

12.8 Why depth, specifically — the hierarchical-features story

A common misconception: *'a single perceptron is logistic regression, so a deep network is just stacked logistic regressions — depth therefore adds nothing fundamentally new.'* This is wrong for one crucial reason: the activation between layers makes the composition **non-linear**. Each layer transforms the previous layer's outputs into **new features**, and those new features live in a richer space than the inputs.

Concretely, in a CNN trained on images:

- **Layer 1** learns simple edges, blobs of colour, oriented gradients.
- **Layer 2** combines those edges into corners, lines, textures, simple shapes.
- **Layer 3** combines shapes into recognisable parts: eyes, wheels, leaves.
- **Layer 4+** assembles parts into objects: faces, cars, plants.

Each level is a non-linear function of the previous level's features, which is something a single (wide-but-shallow) layer simply cannot achieve efficiently. The XOR problem is the smallest illustrative example of this principle: no single perceptron can compute XOR, but a 2-2-1 MLP solves it cleanly because the hidden layer first computes $(x_1 \text{ OR } x_2)$ and $(x_1 \text{ AND } x_2)$, and the output combines them.

Mathematically, certain functions need exponentially fewer parameters in a deep network than a shallow one (Telgarsky, 2016; Bengio & LeCun, 2007). Depth is not 'iterated logistic regression' — depth is **compositional feature learning**, and that is its real superpower.

CONTACT SESSION 13

Deep Learning — Activations, Vanishing Gradient, Regularisation

Deep Learning — Networks With Many Hidden Layers

13.1 What 'deep' actually means

A 'deep' network is, mathematically, just an MLP with more than two or three hidden layers. But that simple shift — going from a few hundred parameters to a few hundred million — surfaces three practical problems we now know how to defeat:

- **Bad activation choices kill gradients.** Sigmoid in deep nets makes the model untrainable.
- **Massive parameter counts overfit.** Even a million training images become 'memorised' rather than learned.
- **Tuning becomes hard.** More layers, more knobs, more ways for things to go wrong.

Each of these has a clean, well-understood fix. This session is the toolbox.

13.2 Activation functions — the intuition

An activation function decides how a neuron's pre-activation is non-linearly transformed before being passed to the next layer. The shape of this function — and especially its *derivative* — determines whether learning is fast, slow, or impossible.

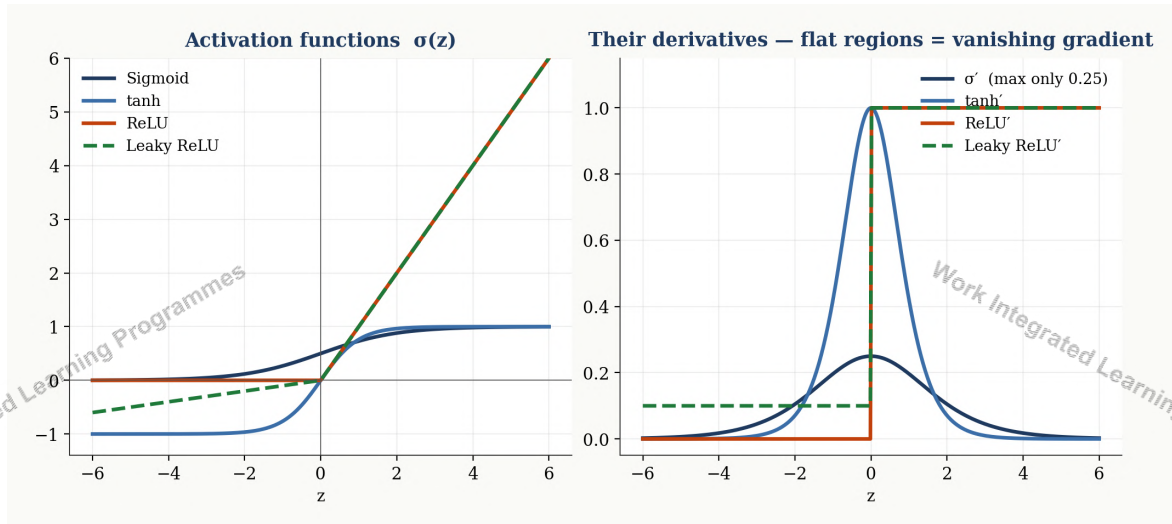


Figure 13.1 — Common activations and their derivatives. The flat parts of σ' are exactly where gradients die during backpropagation.

The five activations to know

Name	Formula	When and why
Sigmoid	$\sigma(z) = 1 / (1 + e^{-z})$	Outputs in (0, 1). Saturates near 0 and 1 — derivative ≤ 0.25 everywhere. Causes vanishing gradients in deep networks. Use only at output of binary classifier.
tanh	$\tanh(z)$	Like sigmoid but zero-centred (range -1 to $+1$). Better than sigmoid in hidden layers but still saturates at the extremes.
ReLU	$\max(0, z)$	Cheap, non-saturating for positive z , sparse activations. Default in modern practice. Drawback: 'dead neurons' — if a neuron is stuck at $z < 0$ throughout training, its gradient is always zero and it never recovers.
Leaky ReLU	$\max(0.01 \cdot z, z)$	A small negative slope keeps neurons alive. ELU and GELU are smoother variants used in modern transformers.
Softmax	$e^{z_i} / \sum_j e^{z_j}$	Used at the output of a multi-class classifier. Turns logits into a valid probability distribution.



Rule of thumb. ReLU (or a variant) for hidden layers. Sigmoid only at the output of a binary classifier. Softmax at the output of a multi-class classifier. tanh occasionally in RNNs and GANs. That is 95% of all practical choices.



Work Integrated Learning Programmes

13.3 The Vanishing Gradient Problem

When we backpropagate through L layers, the gradient is a *product* of L Jacobians — one per layer. If each layer's Jacobian has norm less than 1, their product shrinks exponentially with L . By the time the gradient reaches the early layers it is essentially zero — those layers stop learning, and your fancy deep network behaves like a shallow one.

Concretely: sigmoid's derivative tops out at 0.25. Stack 20 sigmoid layers and the gradient is multiplied by at most $(0.25)^{20} \approx 10^{-12}$. The early layers see no signal.

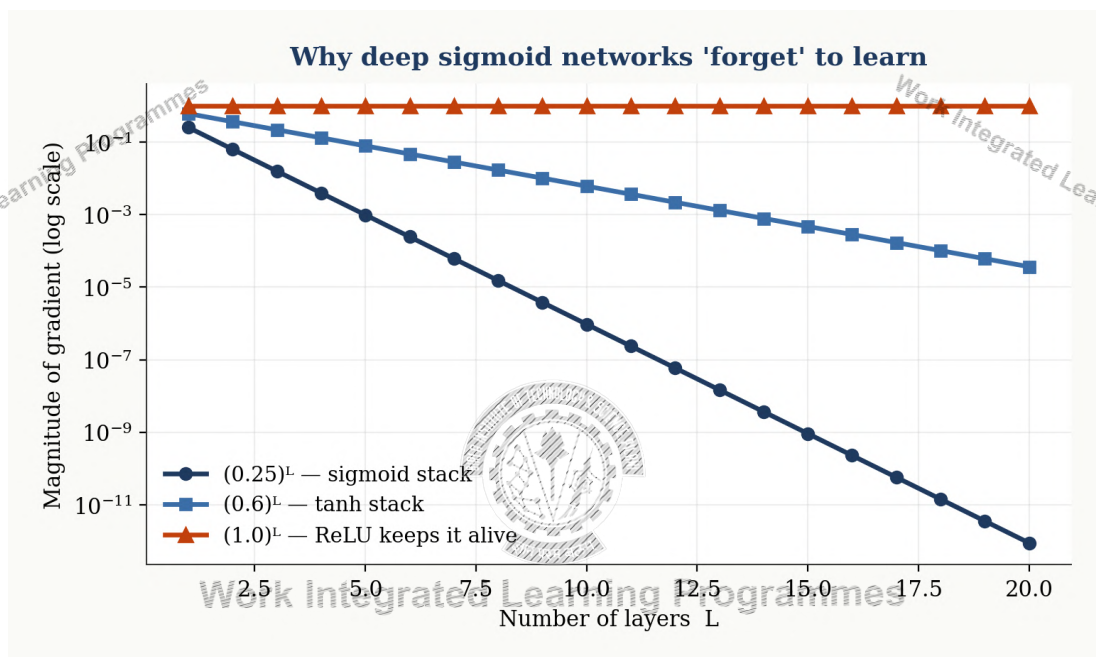


Figure 13.2 — A 20-layer sigmoid stack loses gradient by a factor of 10^{-12} before reaching the first layer. ReLU, with derivative 1 in its active region, keeps it alive.

Why ReLU was a revolution. Its derivative is exactly 1 for positive z . A product of 1s is 1 — gradients flow undiminished through any number of layers. This single change unlocked deep learning at the scale we have today.

Other defences against vanishing gradient

- **Smart weight initialisation** (Xavier for tanh, He for ReLU) so that each layer's outputs have unit variance.
- **Batch normalisation** rescales activations per layer so they don't drift toward saturation.
- **Residual / skip connections** (ResNet) provide a 'shortcut' that lets gradients flow even when a layer's Jacobian is small.
- **Gated architectures** like LSTM and GRU specifically engineer their gradients to flow across many time-steps.

13.4 Regularisation — fighting overfitting

Deep networks have so many parameters that they can memorise the training set perfectly while completely failing on new data. Regularisation adds a small 'cost of complexity' to the loss function so the optimiser is biased toward simpler models.

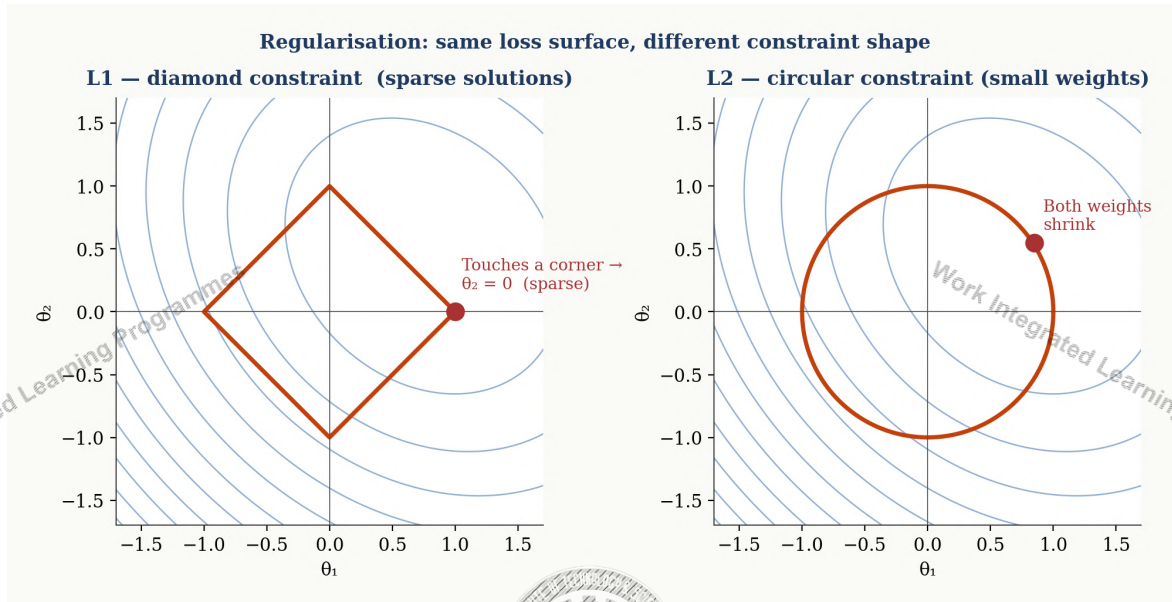


Figure 13.3 — L1 (diamond) tends to push some weights to exactly zero, producing sparse solutions; L2 (circle) just shrinks all weights smoothly.

L2 — Ridge / weight decay

$$L_{\text{total}} = L_{\text{data}} + \lambda \cdot \sum_i w_i^2$$

λ controls 'how much we hate big weights'. The gradient of the penalty is $2 \cdot \lambda \cdot w$, so each step shrinks every weight by a fixed fraction — this is why it is also called 'weight decay'.

L1 — Lasso

$$L_{\text{total}} = L_{\text{data}} + \lambda \cdot \sum_i |w_i|$$

L1's geometry has corners on the axes. Solutions tend to land on a corner — i.e. some weights become *exactly* zero. L1 is therefore a feature-selection mechanism: useless features get killed automatically.

Dropout — the most elegant regulariser ever invented

During training, each neuron is randomly switched off with probability p (typically 0.2 – 0.5). The remaining neurons must learn to be useful even when their teammates are missing — they cannot rely on any one other neuron. At test time all neurons are on but their outputs are scaled by $(1 - p)$ to compensate.

Why this works is mysterious until you realise dropout is roughly equivalent to training an exponentially-large ensemble of sub-networks and averaging them at test time. It is a free ensemble.

Early stopping & data augmentation

- **Early stopping.** Watch the validation loss. The moment it starts going up, stop. The point where training and validation diverge is the onset of overfitting.
- **Data augmentation.** The cheapest regulariser of all: just create more data using sensible transformations — rotate, flip, crop images; insert synonyms in text; add noise to audio. The model must learn the underlying signal, not memorise specific instances.

Whole-session takeaway. A 'deep neural network' is simply an MLP with many hidden layers. Adding depth gave us hierarchical feature learning, but brought along three real problems we now know how to fix:

- **vanishing gradients** → ReLU, BatchNorm, residual connections.
 - **overfitting** → L1/L2 weight decay, dropout, early stopping, data augmentation.
- **tuning** → sensible defaults: Adam optimiser, ReLU, BN, lr = $1e-3$.



Work Integrated Learning Programmes

CONTACT SESSION 14

Deep Networks — CNNs, RNNs, Attention

Convolutional, Recurrent and Attention-Based Networks

14.1 The motivation

MLPs treat every input as a flat, unstructured vector. But real data almost always has structure. Images have *spatial* structure — a pixel cares about its neighbours, not its diagonal opposite. Text has *sequential* structure — a word's meaning depends on the words around it. CNNs and RNNs are MLPs adapted to those two kinds of structure. Attention is the modern, general-purpose glue that beats both.



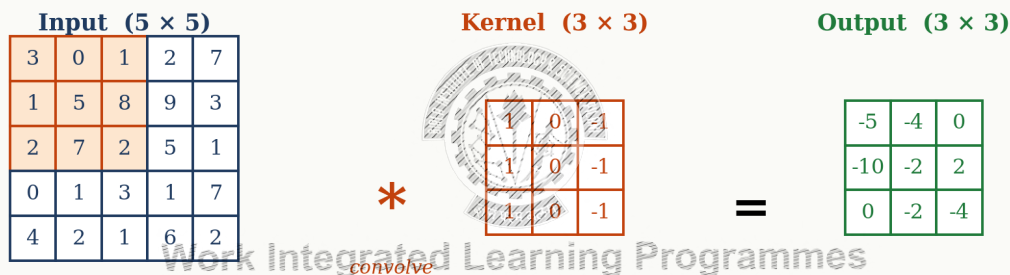
Work Integrated Learning Programmes

14.2 Convolutional Neural Networks (CNNs)

One sentence. A CNN replaces fully-connected layers with *convolutions* — small filters that slide over the image and look for local patterns, sharing weights everywhere they go.

Why convolutions — two big intuitions

- **Translation equivariance.** A cat is a cat no matter which corner of the photo it sits in, so the same filter should fire wherever it sees cat-like pixels. A fully-connected layer cannot enforce this; convolution does it for free.
- **Parameter sharing.** Instead of learning a separate weight for every (input pixel, output pixel) pair, we learn one tiny filter and reuse it across the entire image. That brings parameter count from millions to thousands.



Top-left output cell = $3 \cdot 1 + 0 \cdot 0 + 1 \cdot (-1) + 1 \cdot 1 + 5 \cdot 0 + 8 \cdot (-1) + 2 \cdot 1 + 7 \cdot 0 + 2 \cdot (-1) = -5$
(slide the kernel one cell to the right and repeat — that fills the next output cell)

Figure 14.1 — A 3×3 vertical-edge-detector kernel convolved over a 5×5 input gives a 3×3 output. Each output cell is one dot product.

The math of one convolution cell

Given an input I and a kernel K of size $k \times k$, the output at position (i, j) is:

$$O(i, j) = \sum_{u=0..k-1} \sum_{v=0..k-1} K(u, v) \cdot I(i + u, j + v)$$

In English: place the kernel on top of the image at position (i, j) ; element-wise multiply; sum everything. That single number becomes the output at (i, j) . Slide the kernel by 'stride' cells and repeat.

Output-size formula — the most important formula in CNNs

CNN Output-Size Formula

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

W input width K kernel size P padding S stride

Use 'same' padding ($P = (K-1)/2$, $S = 1$) to keep size unchanged.

Figure 14.2 — The output-size formula for a single convolution layer.

$O = \lfloor (W - K + 2P) / S \rfloor + 1$ where W = input width, K = kernel size, P = padding, S = stride.

Six solved examples — get fluent with the formula

Configuration	Computation	Output size
Input 32×32 ; K=3 ; P=0 ; S=1	$(32 - 3 + 0) / 1 + 1 = 30$	30×30
Input 32×32 ; K=3 ; P=1 ; S=1	$(32 - 3 + 2) / 1 + 1 = 32$	32×32 (same padding)
Input 32×32 ; K=5 ; P=2 ; S=1	$(32 - 5 + 4) / 1 + 1 = 32$	32×32
Input 32×32 ; K=3 ; P=0 ; S=2	$\lfloor (32 - 3 + 0) / 2 \rfloor + 1 = 15$	15×15
Input 224×224 ; K=7 ; P=3 ; S=2	$\lfloor (224 - 7 + 6) / 2 \rfloor + 1 = 112$	112×112 (typical first conv in ResNet)
Input 7×7 ; K=3 ; P=1 ; S=2	$(7 - 3 + 2) / 2 + 1 = 4$	4×4

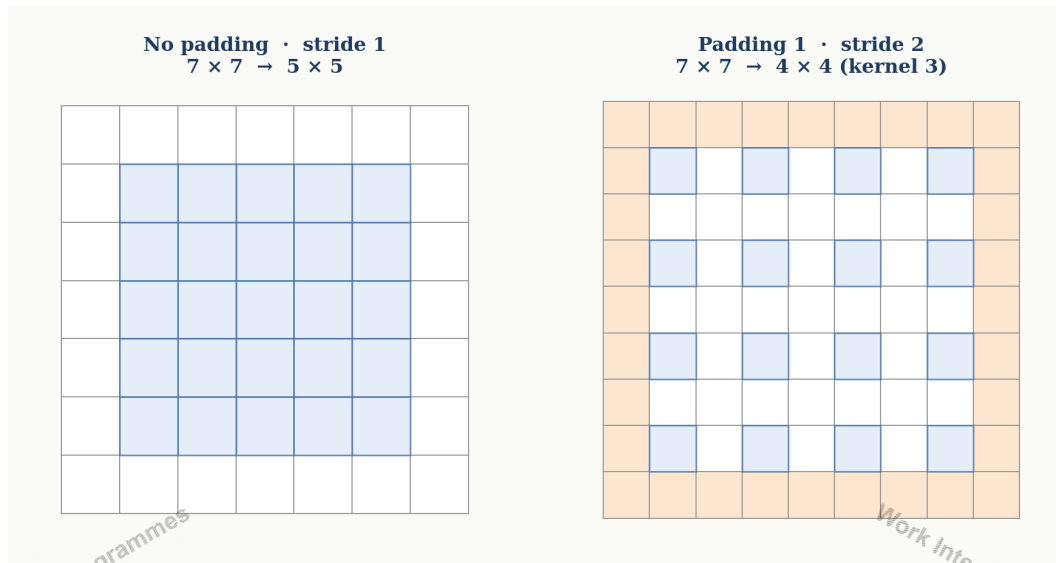


Figure 14.3 — Visual: same kernel, different (padding, stride) → different output sizes. Blue cells are output positions; orange cells are zero-padding.

Pooling — the friendly down-sampler

After a few conv layers we usually *pool*: take a small window (typically 2×2) and replace it with its max (max-pool) or average (avg-pool). Pooling halves spatial dimensions and gives the network a degree of translation *invariance* — small shifts in the input barely change the output.

Max pooling vs average pooling

It is tempting to say 'average pooling preserves more information so it must be better.' That is misleading — the two operations encode different priors about what matters in the window:

- **Max pooling** asks: '*is the feature this filter detects present anywhere in the window?*' Perfect when you care about **presence** over location — has an edge fired here? a corner? a stripe? Max keeps the strongest activation and discards everything else. Used everywhere in classification CNNs (VGG, AlexNet, early ResNets).
- **Average pooling** asks: '*how strong is the feature on average in the window?*' Perfect when you want a smoothed, holistic summary — for instance the very last layer of many modern networks (global average pooling) which collapses a feature map into one number per channel before the final classifier. Average pooling also tends to keep noise smaller (averaging is a low-pass filter).

Rule of thumb. Use **max pooling** in early/mid layers of a classification CNN, where you want to detect 'is this feature anywhere in this region?' Use **average pooling** for global summaries (final layer) and in tasks where smoothness matters more than peak responses (e.g., regression of intensity maps, denoising). Modern architectures often skip pooling entirely and use strided convolutions instead — but understanding the trade-off remains important.

Putting it all together — a tiny CNN classifier

[Conv 32 filters, 3×3, ReLU, BN] → [Conv 32, 3×3, ReLU, BN] → [MaxPool 2×2]
→ [Conv 64, 3×3, ReLU, BN] → [Conv 64, 3×3, ReLU, BN] → [MaxPool 2×2]
→ [Flatten] → [Dense 128, ReLU, Dropout] → [Dense 10, Softmax]

That recipe — alternating conv/conv/pool, doubling channels, ending in a small dense head — works astonishingly well on any image-classification task and is essentially how every famous CNN (LeNet, AlexNet, VGG, ResNet) is built.

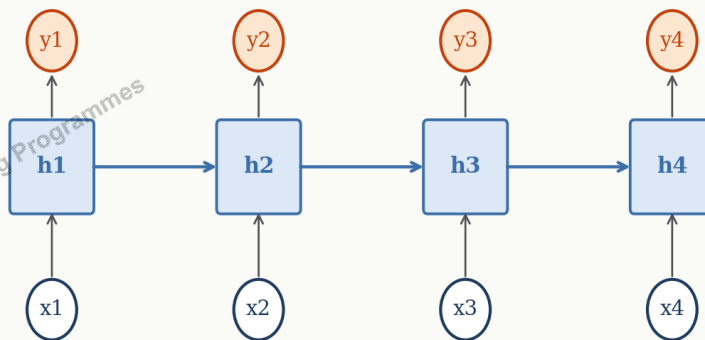


Work Integrated Learning Programmes

14.3 Recurrent Neural Networks (RNNs)

One sentence. An RNN reads one element of a sequence at a time, carrying a 'hidden state' that summarises everything it has seen so far.

Recurrent Neural Network — unrolled in time



The same weights W are reused at every time step → 'memory' of the past

Figure 14.4 — An RNN unrolled across time. The same weights are reused at every time step.

The recurrence equations

$$h_t = \sigma(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b)$$

$$y_t = \sigma(W_{hy} \cdot h_t + c)$$

The hidden state h_t is the network's memory: it depends on the current input x_t AND the previous hidden state h_{t-1} . Crucially, the weight matrices W_{xh} , W_{hh} , W_{hy} are *shared at every time step* — that is what makes this **recurrent**, and what lets a single model process sequences of any length.

LSTM and GRU — solving long-term dependency

Plain RNNs struggle to remember anything from many steps ago — backprop through time involves multiplying many Jacobians, and the gradient vanishes (or explodes).

LSTMs (Long Short-Term Memory) introduce a separate *cell state* c_t with three gates that decide what to *forget*, what to *add*, and what to *output*:

$$\begin{aligned} f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \text{ (forget gate)} \\ i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \text{ (input gate)} \\ o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \text{ (output gate)} \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \\ h_t &= o_t \odot \tanh(c_t) \end{aligned}$$

The cell state c_t is a *highway* — information can flow across many time steps with only multiplicative gating, not the repeated matrix multiplications that make plain RNNs forget. GRUs are a smaller cousin with two gates instead of three; usually similar accuracy with fewer parameters.



Work Integrated Learning Programmes

14.4 Encoder - Decoder Architectures (seq2seq)

Setup. Many sequence problems map one sequence to another of different length: machine translation, summarisation, speech recognition. An *encoder* RNN/LSTM reads the source sequence and packs it into a single context vector c . A *decoder* RNN/LSTM uses c (and what it has produced so far) to generate the target one token at a time.

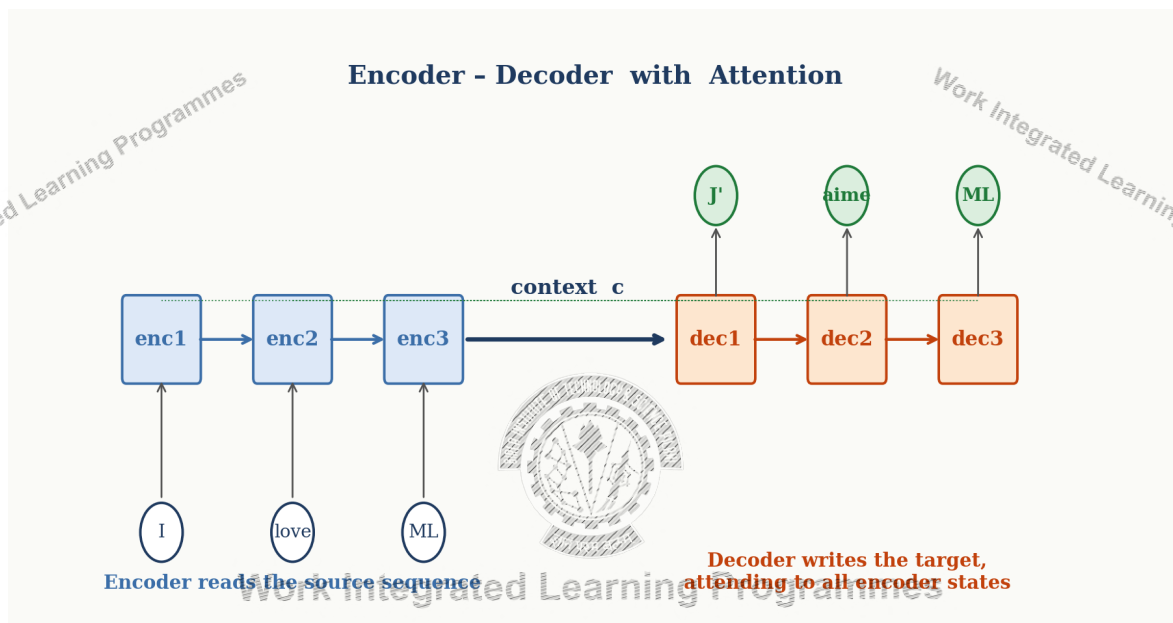


Figure 14.5 — The encoder-decoder architecture with attention. The decoder peeks at every encoder hidden state, not just the last one.

The bottleneck problem

Squashing an entire 50-word sentence into one fixed-length vector c is an unreasonable demand. Translations of long sentences degrade rapidly. The fix: **attention**.

14.5 The Attention Mechanism

The idea. At each decoding step, instead of being handed a single c , the decoder gets to look back at *all* encoder hidden states and form a *weighted* average of them — paying more attention to the encoder positions that are most relevant *right now*.

The math, in three lines

$$\text{score}(s, h_j) = s^T \cdot W \cdot h_j \text{ (any compatibility function works)}$$

$$\alpha_j = \text{softmax}_j(\text{score}(s, h_j)) \leftarrow \text{attention weights}$$

$$c_t = \sum_j \alpha_j \cdot h_j \leftarrow \text{the dynamic context vector}$$

where s is the decoder's current state and h_j is the j -th encoder hidden state. The α_j values sum to 1 and tell us, in plain English, 'right now, decoder, look 70% at source word 3 and 30% at source word 7'.

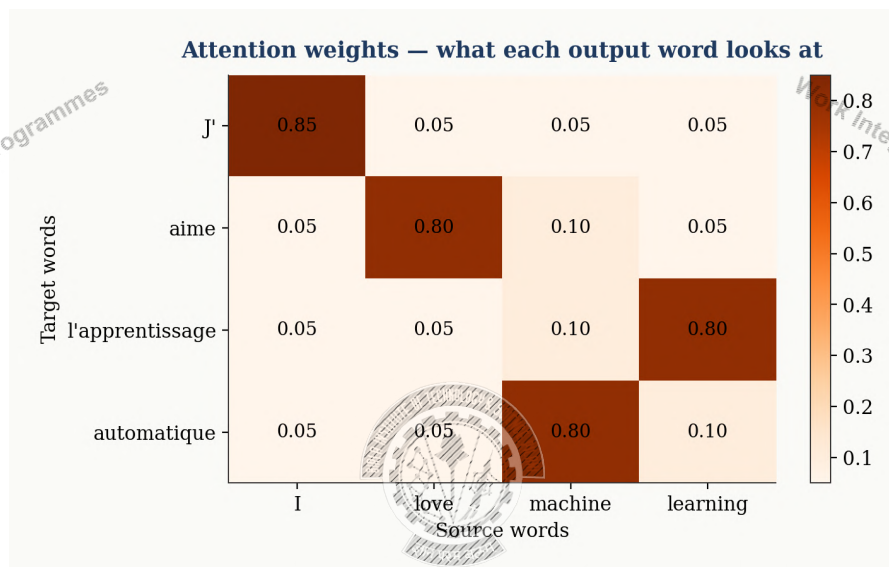


Figure 14.6 — Attention weights for a French translation. The decoder learns the alignment between source and target words automatically.

Why attention exploded into Transformers. If attention is so powerful, why bother with the recurrent backbone at all? That single insight gave us the **Transformer** (Vaswani et al., 2017), where attention is the *entire* model — a stack of self-attention layers, no recurrence, no convolutions. Every modern LLM (BERT, GPT, Claude, Llama, Gemini) descends directly from this idea.

CONTACT SESSION 15

FAccT — Fairness, Accountability, Transparency

FAccT — Bias, Fairness and Interpretability

15.1 Why this matters now

Most of this guide has been about getting models to be *accurate*. This session asks a harder question: are they also *fair* and *understandable*? Models now decide who gets a loan, who is released on bail, what news you read, what ad you see, what résumés are forwarded to a hiring manager. Accuracy alone is no longer the right objective — sometimes it is the *wrong* objective.

FAccT — short for Fairness, Accountability and Transparency — is the field that puts a moral and operational lens on top of machine learning. Two big sub-questions matter most for engineers and instructors:

- **Bias and fairness** — does our model work equally well for every group it touches?
- **Interpretability** — can we explain a model's decision in language a human (or regulator, or end user) can audit?

15.2 Where bias comes from

ML systems do not invent bias from nothing. They *amplify* bias that was already in the data, the labels, or the deployment context. It helps to name the sources:

- **Historical bias** — past data reflects unjust patterns (who got hired in the 1990s). The model learns to copy them.
- **Representation bias** — some groups are under-represented in training data, so the model is simply *worse* on them.
- **Measurement bias** — the proxy you measure isn't what you actually want (using 'past arrests' as a proxy for 'crime risk' bakes in policing biases).
- **Aggregation bias** — a single model is forced to fit groups that behave differently.
- **Evaluation bias** — average accuracy hides poor performance on minorities.
- **Deployment bias** — the model is used in a context different from its training distribution.

15.3 Fairness — there is no single definition

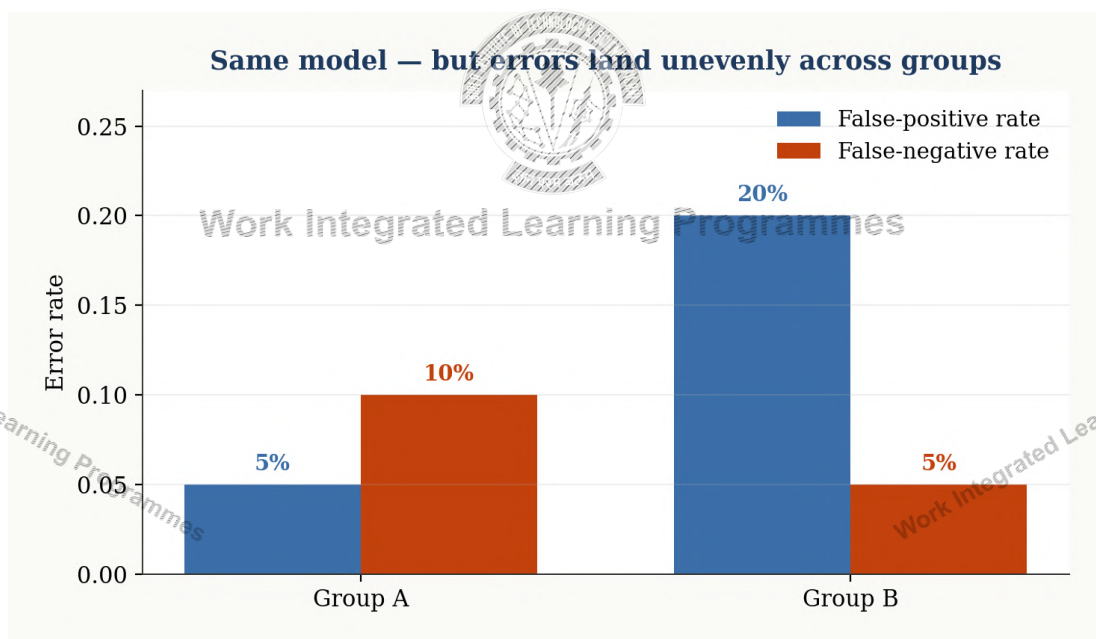


Figure 15.1 — Same model — but errors land unevenly across groups. That uneven landing is the visual definition of unfairness.

Several reasonable mathematical notions of fairness exist. Famously they are **provably incompatible** in general — you cannot satisfy all of them at once. Pick the definition your stakeholders agree with, then measure it explicitly.

Criterion	Formal definition	Plain English
Demographic Parity	$P(\hat{y} = 1 \mid A = a)$ is the same for every protected group a	Selection rate is equal across groups.
Equalised Odds	$P(\hat{y} = 1 \mid y = y, A = a)$ same for every (y, a)	False-positive AND false-negative rates are equal across groups.
Equal Opportunity	$P(\hat{y} = 1 \mid y = 1, A = a)$ same across groups	Among the people who deserve a 'yes', the model says 'yes' equally often.
Calibration	$P(y = 1 \mid \hat{y} = p, A = a) = p$ for every group	When the model says '70%', it really is 70% — for every group.

Impossibility result (Chouldechova, 2017). Except in trivial cases, no model can simultaneously satisfy **calibration** AND **equalised odds** across groups when base rates between those groups differ. Fairness is therefore a value judgement, not a math identity. Engineers must choose, transparently, which fairness criterion they will optimise for — and why.

15.4 Mitigating bias

- **Pre-processing** — re-balance the dataset (oversample under-represented groups, re-weight examples, learn fair representations).
- **In-processing** — add a fairness constraint or penalty to the loss function during training.
- **Post-processing** — adjust the decision threshold per group to equalise the chosen fairness metric.

There is no automatic right answer — every choice trades off accuracy and different fairness criteria. The point of FAcCT is to make these tradeoffs explicit and accountable.

15.5 Interpretable Machine Learning

Interpretability is the degree to which a human can understand the cause of a model's decision. We want it for: **trust, debugging, auditing, scientific discovery**, and to satisfy laws like GDPR's 'right to explanation'.

Two families of methods

- **Intrinsic interpretability** — use a model that is itself simple. Linear regression, small decision trees, rule lists, generalised additive models. Cheap to explain, but accuracy ceiling is real.
- **Post-hoc interpretability** — fit a complex model and *then* explain it. Feature importance, permutation importance, partial-dependence plots, LIME, SHAP, integrated gradients.

LIME — local linear approximation

LIME (Local Interpretable Model-agnostic Explanations) explains a single prediction by perturbing the input slightly, asking the black box model for predictions on each perturbation, and fitting a tiny linear model on those perturbations. The linear model's coefficients are the local explanation. It says: 'in the neighbourhood of *this* input, the model behaves linearly with these weights.'

SHAP — fair credit via game theory

SHAP (SHapley Additive exPlanations) borrows from cooperative game theory. Each feature is a 'player' contributing to the final prediction; SHAP gives every feature its fair share of the credit, computed as the average marginal contribution over all possible orderings of features:

$$\phi_j = \sum_{S \subseteq N \setminus \{j\}} \left[\frac{|S|! \cdot (|N| - |S| - 1)!}{|N|!} \right] \cdot [v(S \cup \{j\}) - v(S)]$$

It is the *only* attribution method that satisfies four desirable properties simultaneously — efficiency, symmetry, dummy and additivity — properties that make the explanations trustworthy and consistent.

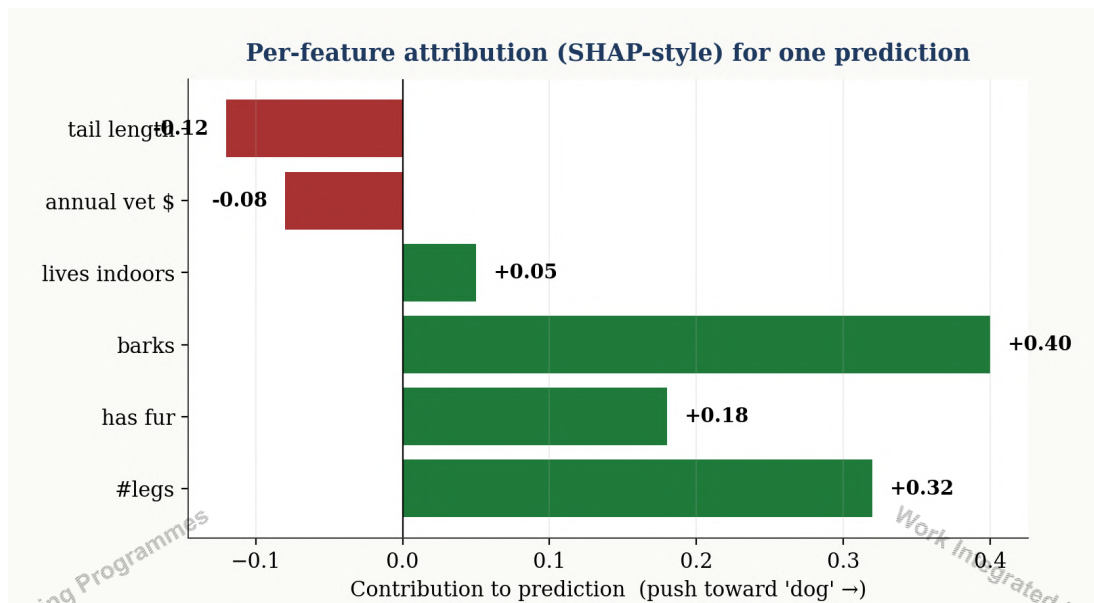


Figure 15.2 — A SHAP-style attribution for one prediction. Green bars push the prediction toward 'dog'; red bars push it away. Length = magnitude of contribution.

Other useful tools you should know

- **Permutation importance** — shuffle a feature; how much does accuracy drop? Big drop = important feature.
- **Partial-dependence plot (PDP)** — vary one feature, average predictions over all other features. Shows the marginal effect of the chosen feature.
- **Counterfactual explanations** — “what would have to change about my application for the loan to be approved?” Often the most actionable type of explanation.
- **Saliency maps** (for images) and **integrated gradients** — highlight the pixels (or words) most responsible for the prediction.

15.6 Interpretability in high-stakes domains

It is tempting to argue that the *most accurate* model should always win. In medicine, finance, criminal justice and other regulated domains, that argument is usually wrong. Several considerations push back against pure-accuracy thinking:

- **Accountability.** When a hospital is sued for a missed sepsis call, the doctor must defend the decision. *'The model said so'* is not a defence; *'the model said so because of these specific patient features, and here is the supporting literature'* is.
- **Distribution shift.** A complex black-box that scored a brilliant AUC on retrospective data may quietly fail when the patient population, the test panel, or even the data-collection pipeline changes. An interpretable model lets clinicians spot when assumptions break.

- **Trust and adoption.** Doctors will not deploy what they cannot reason about — even a 3% AUC gain is worthless if the tool sits unused.
- **Regulatory compliance.** GDPR Article 22 grants individuals a right to meaningful information about automated decisions affecting them. The FDA's 2023 guidance on AI/ML in medical devices similarly demands transparency.
- **Bias detection.** Interpretable models surface when a system is using a sensitive proxy (race, ZIP code, insurance type) it shouldn't be using. A black box hides such failure modes until they cause harm.

On the other hand, accuracy *does* matter — particularly when the better model genuinely catches more lives or detects more fraud. The right question is not 'accurate vs interpretable' but: *how much accuracy are we willing to trade for how much explainability — and can we get the best of both?*

Pragmatic recipe for high-stakes ML.

1. Start with the most interpretable model that meets the accuracy bar (often a regularised linear model or a small tree).
2. If accuracy is insufficient, escalate to a forest or gradient-booster — still interpretable via SHAP / permutation importance.
3. Only escalate to a deep model if the gap is large and explanations can be provided post-hoc (LIME, SHAP, integrated gradients).
4. Run the deep model side-by-side with the simpler model and audit disagreements — that is where bias and distribution shift surface.
5. In regulated domains, document the chosen tradeoff explicitly: *'we accept a 0.03 AUC drop to keep decisions explainable in clinical review.'*

Final word. Accuracy is necessary but not sufficient. A model that is wildly accurate yet discriminates against people, or that we cannot question when it errs, is not a model we should deploy. The most valuable engineers of the next decade will be the ones who can *build* AND *audit* such systems with equal rigour. That is what this session has been about, and what FAccT is about.

Closing Notes for Students

You have now seen the second half of Applied Machine Learning end to end — from the simplest decision tree to deep convolutional, recurrent and attention-based architectures, ending with the social and ethical lens that should sit on top of all of it.

Three pieces of advice as you study, build labs, and walk into your exams:

- **1. Build the picture before the equations.** Every concept here was first a sketch on a notebook page. If you can draw it, you understand it. If you can only quote the formula, you don't yet.
- **2. Run the code yourself.** Read the Python notebooks for every algorithm in this guide. Implement at least one of them from scratch (decision tree, K-Means, and a tiny MLP are excellent practice).
- **3. Ask the 'why'.** Why do we need a non-linearity? Why does L1 give sparsity and L2 just shrinkage? Why does attention beat fixed context? The exam will reward students who can answer these — and so will every interview you ever sit.

If you spot an error in this guide — a typo, a wrong number, an unclear sentence — please tell me on the eLearn portal. I update this document every semester based on student feedback.

Best of luck for your exams and your projects.
— **Saurabh**

— **End of Companion Guide** —